



Starfish optimization algorithm (SFOA): a bio-inspired metaheuristic algorithm for global optimization compared with 100 optimizers

Changting Zhong^{1,2} · Gang Li^{2,3} · Zeng Meng^{4,5} · Haijiang Li⁶ · Ali Riza Yildiz⁷ · Seyedali Mirjalili^{8,9}

Received: 22 November 2023 / Accepted: 7 October 2024 / Published online: 16 December 2024
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2024

Abstract

This work presents the starfish optimization algorithm (SFOA), a novel bio-inspired metaheuristic for solving optimization problems, which simulates behaviors of starfish, including exploration, preying, and regeneration. SFOA consists of two main phases of exploration and exploitation. The exploration phase mimics the explorative behavior of starfish by the hybrid search pattern of combining with the five-dimensional and unidimensional search patterns to increase the computational efficiency and ensure the search capacity. The exploitation phase simulates the preying and regeneration behaviors of starfish, with a two-directional search strategy and special movement, to ensure convergence in exploitation. This work validates SFOA's performance on 65 benchmark functions from classical functions, CEC 2017 and CEC 2022 test suites, and compares with 100 different metaheuristic algorithms, including state-of-the-art optimizers, such as marine predators algorithm, water flow optimizer (WFO), LSHADE, LSHADE-cnEpSin, and LSHADE-SPACMA. Statistical results from one-on-one comparisons demonstrate that the proposed SFOA outperforms 95 compared algorithms in accuracy and 97 algorithms in efficiency, which is only worse than WFO both in accuracy and efficiency. The scalability analysis also demonstrates that SFOA has the capacity to solve high-dimensional benchmark functions. Furthermore, ten real-world engineering optimization problems illustrate the effectiveness of SFOA to achieve global solutions and exhibit stable results. In conclusion, SFOA is promising for solving various optimization problems. The source code of SFOA is publicly available at: <https://ww2.mathworks.cn/matlabcentral/fileexchange/173735-starfish-optimization-algorithm-sfoa>.

Keywords Metaheuristic · Starfish optimization algorithm · Water flow optimizer · High-performance · Optimization · Particle swarm optimization

1 Introduction

Due to the rapid development of scientific and technological achievements, artificial intelligence techniques have become ubiquitous in modern society. Optimization is a necessary component of artificial intelligence due to its capacity of determining the best solution from a variety of possible options [1]. Typically, an optimization problem involves three fundamental elements: objective function(s), design variables, and constraints, which necessitates the optimization method to achieve the global solution [2]. Before the advent of metaheuristic algorithms, deterministic optimization algorithms were used to be the only choice for optimization problems, which may struggle to avoid local optimum in solving highly nonlinear optimization problems without the introduction of randomness.

In addition, gradient information derived from objectives and constraints is typically required for deterministic optimization algorithms, which may be challenging in solving complex optimization problems. Inspired by natural behaviors, metaheuristic algorithms as a stochastic optimization technique have gained popularity in the optimization fields due to their numerous benefits, such as population-based mechanisms with various candidate solutions, easy to implement, highly flexible, not requiring gradient information, black-box processes only with inputs and outputs. Thus, metaheuristic algorithms are very appropriate for optimization problems, such as feature selection [3], scheduling [4], civil engineering [5], and mechanical design [6].

Numerous metaheuristic algorithms have been developed by researchers and engineers by now, including particle swarm optimization (PSO) [7], genetic algorithm (GA) [8], and differential evolution [9], which have been

Extended author information available on the last page of the article

widely applied to solving different optimization problems. Extracted from these classical metaheuristic algorithms, several advanced algorithms such as ensemble PSO [10], non-dominated sorting genetic algorithm (NSGA-II) [11] and SHADE [12] were presented, usually outperformed the original metaheuristic algorithms. Besides, many advanced metaheuristic algorithms have gained considerable attention, including biogeography-based optimization [13], grey wolf optimizer [14], teaching–learning-based optimization [15], Harris Hawks optimization [16], arithmetic optimization algorithm [17], and so on. Section 2 provides a comprehensive review of the novel metaheuristic algorithms.

Although metaheuristics have different performance and inspirations, how to balance exploration and exploitation is still a common challenge. The exploration denotes the global search capacity of search agents, while the exploitation relates to the local search ability. If the balance between exploration and exploitation is not achieved in a metaheuristic algorithm, it may fall into local stagnation or premature convergence. It is worth noting that any metaheuristic algorithm relies on stochastic search, and thus, it cannot ensure global convergence for all optimization problems. This characteristic was illustrated by the no free lunch (NFL) theory [18], which motivates researchers to develop more efficient and accurate metaheuristic algorithms. However, as Sörensen pointed out [19], several novel metaheuristic algorithms have been developed from the slightly changes on the previous metaheuristic algorithms, and the metaphors are based on the existing mathematical models. One of the main concerns in this field is that the novel theoretical mathematical model should be developed, such as the searching mechanism in the exploration and exploitation. Another concern is that a thorough experiment is not usually implemented, which may not ensure the reliability and effectiveness of the novel metaheuristic algorithm. By conducting rigorous experiments and detailed comparisons of algorithms, researchers can find the strengths and weaknesses of the proposed metaheuristic algorithm and identify the limitations or potential shortcomings, which is imperative in developing novel metaheuristic algorithms for optimization.

In this study, we present a novel metaheuristic algorithm called the starfish optimization algorithm (SFOA), inspired by the preying, exploration and reproduction behaviors of starfish. According to the authors' knowledge, it is the first time to establish the mathematical model of starfish for optimization problems. Starfish exhibit exploration, preying, and regeneration behaviors that we model as the exploration and exploitation phases of the proposed SFOA algorithm. In validation, we first conduct an experiment with 65 benchmark functions, including 24 classical functions, 29 CEC 2017 test functions, and 12 CEC 2022

test functions. In the numerical experiment, SFOA is compared with totally 100 different metaheuristic algorithms, including four algorithms proposed before 2020, 13 algorithms developed in 2000–2009, 29 algorithms in 2010–2019, and 54 algorithms since 2020, that can comprehensively investigate the performance of the proposed SFOA algorithm in solving different optimization problems. SFOA is also verified by the sensitivity and scalability analysis, including high-dimensional optimization problems. Furthermore, ten real-world optimization problems are used to verify its effectiveness.

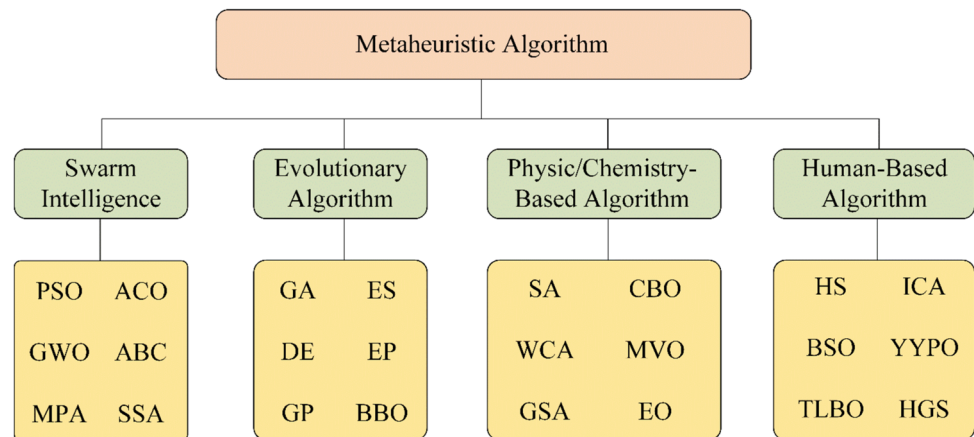
The contributions of this work in the metaheuristic field are summarized as follows:

1. A novel metaheuristic algorithm called the starfish optimization algorithm (SFOA) is inspired by the starfish for global optimization and engineering problems. SFOA includes the exploration and exploitation phases, simulating the behaviors of searching, preying, and regeneration.
2. In the exploration of SFOA, the hybrid search pattern is employed in SFOA by combining the five-dimensional search pattern for the optimization problem with $D > 5$ and the unidimensional search pattern for $D \leq 5$ in the exploration phase, while in the exploitation phase, SFOA introduces the two parallel two-directional search strategy to simulate the preying behaviors and constructs the regeneration strategy to enhance the global convergence.
3. This work systematically investigates the performances of the proposed SFOA and 100 comparing algorithms in solving widespread benchmark functions. The statistical results indicate that SFOA outperforms 95 algorithms in accuracy and 97 algorithms in efficiency, illustrating that SFOA performs well for global optimization. SFOA is also validated in 10 real-world engineering optimization problems.

The structure of this paper is organized as follows. The literature review is introduced in Sect. 2. The theory of SFOA is illustrated in Sect. 3. Sections 4 and 5 present the experiments of benchmark functions and real-world optimization problems, respectively. Finally, the conclusions and future works are summarized in Sect. 6.

2 Related works

This section gives a comprehensive review of the novel metaheuristic algorithms. According to the inspiration in nature, the metaheuristic algorithms can be mainly divided into four components, namely swarm intelligence, evolutionary algorithm, physics/chemistry-based algorithm, and human-inspired algorithm, as shown in Fig. 1.

Fig. 1 Inspiration of metaheuristics

The first category in the metaheuristic field is swarm intelligence, which simulates the preying or swarm behaviors of creatures in nature. One of the most popular swarm intelligence algorithms is PSO, which was inspired by the swarming behaviors of fish and birds. The updating rule of PSO has three phases: the momentum part for control velocity, the cognitive part for local search, and the social components for global search. Although PSO is a simple and efficient algorithm, it may converge slowly for complex optimization problems, and it is also sensitive to parameter tuning. Thus, researchers developed abundant enhanced PSO algorithms to solve different optimization problems, as summarized in [20, 21]. The ant colony optimizer (ACO) [22] was inspired by the behavior of the ant system, using the release of pheromones and movement along paths with high concentrations of pheromones. Karaboga [23] developed the artificial bee colony (ABC) to mimic the behaviors of honeybees, including employed bees, onlooker bees, and scout bees. Gray wolf optimizer (GWO) was proposed by Mirjalili et al. [14] in 2014, inspired by the hierarchy and preying behaviors of gray wolves, where the alpha wolf leads other wolves to hunt prey, and different attacking strategies were introduced in the algorithm. Mirjalili et al. [24] also proposed the salp swarm algorithm, inspired by the leading preying and circling behavior of salps in marine. Another example is Harris hawks optimization (HHO) [16], mimicking the preying and hunting behaviors of Harris hawks, with four strategies in the attacking phase. Faramarzi et al. [25] proposed the marine predator algorithm (MPA) for solving metaheuristic algorithms simulating the Brownian and Levy trajectories. Zhong et al. [26] proposed the beluga whale optimization (BWO), inspired by the swimming, preying and whale fall behaviors of beluga whales, which has good convergence for high-dimensional optimization problems. Other swarm intelligence algorithms that have shown their effective for optimization problems include shuffled frog-leaping algorithm [27], firefly algorithm [28],

cuckoo search [29], bat algorithm [30], flower pollination algorithm [31], krill herd [32], artificial algae algorithm [33], moth-flame optimization [34], whale optimization algorithm [35], spotted hyena optimizer [36], seagull optimization algorithm [37], sailfish optimizer [38], bald eagle search [39], chimp optimization algorithm [40], manta ray foraging optimizer [41], slime mould algorithm [42], tunicate swarm algorithm [43], aquila optimizer [44], carnivorous plant algorithm [45], chameleon swarm optimization [46], golden eagle optimizer [47], horse herd optimization [48], red fox optimization algorithm [49], artificial hummingbird algorithm [50], artificial rabbits optimization [51], cheetah optimizer [52], dandelion optimizer [53], dwarf mongoose optimization [54], golden jackal optimization [55], gannet optimization algorithm [56], honey badger algorithm [57], Komodo Mlipir algorithm [58], mountain gazelle optimizer [59], orca predation algorithm [60], prairie dog optimization [61], reptile search algorithm [62], snake optimizer [63], starling murmuration optimizer [64], white shark optimizer [65], crayfish optimization algorithm [66], sand cat swarm optimization [67], termite life cycle optimizer [68], and so on.

The second category of metaheuristic algorithms is evolutionary algorithms, typically inspired by the evolutionary process in nature. Genetic algorithm (GA) proposed by Holland [8] is a well-known evolutionary algorithm applied to various optimization problems. Different evolution (DE) is another variant of GA with a mutation and crossover phase, proposed by Storn and Price [9]. This algorithm has several high-performance variants, such as SHADE [12, 69] and LSHADE-cnEpSin [70]. Simon [13] proposed biogeography-based optimization to mimic the immigration and emigration strategies of species in neighborhood habitats. Other evolutionary algorithms consist of evolution strategy [71], genetic programming [72], evolutionary programming [73], barnacle mating optimizer [74], water flow optimizer (WFO) [75], Bayesian-evolutionary algorithm [76, 77], etc.

The third category of metaheuristic algorithms is mainly inspired by physics- or chemistry-based phenomena in nature. Simulated annealing was inspired by the cooling and annealing process of metals [78]. Erol and Eksin [79] developed the big bang-big crunch algorithm to simulate the big bang theory of the universe, including the big bang phase of particles losing energy to move, and the big crunch phase of particles converging to specific directions. Rashedi et al. [80] proposed the gravitational search algorithm, inspired by the law of gravity and following the Newton's gravitational force to mimic the moving behavior of plants. Kaveh and Talatahari [81] developed the charged system search to mimic the behaviors of Coulomb law of electrostatics and Newton's law of mechanics. Eskandar et al. [82] introduced the water cycle algorithm to mimic the water cycle process of the flowing behaviors of rivers and streams converging to the sea. Faramarzi et al. [83] proposed the equilibrium optimizer to simulate how to obtain the equilibrium states from volume mass balance models. In this category, other physics/chemistry-based metaheuristic algorithms consist of invasive weed optimization [84], fireworks algorithm [85], ray optimization [86], colliding bodies optimization [87], stochastic fractal search [88], multi-verse optimizer [89], sine cosine algorithm [90], water evaporation algorithm [91], thermal exchange optimization [92], atom search algorithm [93], atomic orbital search [94], artificial ecosystem optimization [95], dynamic differential annealed optimization [96], Lévy flight distribution [97], Lichtenberg algorithm [98], homonuclear molecules optimization [99], seasons optimization algorithm [100], special relativity search [101], Fick's law algorithm [102], RIME optimization algorithm [103], Young's double-slit experiment optimizer [104], and so on.

In the last category, human-based algorithms are usually inspired by human or social group behaviors. Harmony search algorithm was proposed by Geem and Kim [105] to mimic the improvisation of music players and sounds, such as fiddle, saxophone, and keyboard. Shi [106] proposed the brainstorming optimization to imitate human brainstorming behaviors in meetings, where clustering measures the distribution of cluster centroids and information entropy of population. Imperialist competitive algorithm simulates the generation and death behaviors of empires and colonies [107]. Rao et al. [15] proposed teaching–learning-based optimization (TLBO) to simulate the teaching and learning behaviors of students in the classroom, which has good performance for various optimization problems. Rao [108] also developed a simple JAYA algorithm with good efficiency for optimization problems, with several developments such as enhanced JAYA (EJAYA) algorithm [109]. Yin-Yang-pair optimization (YYPO) [110] was inspired by the dualities called Yin and Yang, two complementary and

interdependent extremes from Chinese philosophy, where the algorithm has the splitting stage and archive stage to control the exploration and exploitation. Other algorithms in this category are culture algorithm [111], league championship algorithm [112], soccer league competition algorithm [113], socio-evolution and learning optimization [114], supply–demand-based optimization [115], fitness distance balance [116], gradient-based optimizer [117], group teaching optimization algorithm [118], heap-based optimizer [119], chaos game optimization [120], cooperation search algorithm [121], hunger game search [122], RUN [123], Ali Baba and the forty thieves [124], alpine skiing optimization [125], human felicity algorithm [126], and so on. The list of novel metaheuristic algorithms is tabulated in Table 1. A more comprehensive investigation of nature-inspired metaheuristic algorithms can be found in [127].

3 Starfish optimization algorithm

Based on the behaviors of starfish, we propose a novel swarm intelligence algorithm called the starfish optimization algorithm (SFOA) to solve global and engineering optimization problems. This section introduces the inspiration, the mathematical model, the procedure and the complexity of SFOA. Finally, the comparison between SFOA and other popular metaheuristic algorithms is also illustrated.

3.1 Inspiration

Starfish, also known as sea stars, are marine invertebrates, which belong to the echinoderms in the class Asteroidea. With over 2000 species worldwide, starfish typically have five arms with a central disk, giving them a distinctive star-like appearance. Some other species of starfish have seven to more than ten arms. Their diameters range from 0.5 mm to over 50 cm, typically between 12 and 25 cm [128]. Starfish are distributed throughout the world's oceans, primarily in the deep sea, and rarely found in freshwater areas due to their lack of an osmoregulation system. Most starfish have a lifespan of about ten years, but some can live up to 35 years. Figure 2 displays several photographs of starfish.

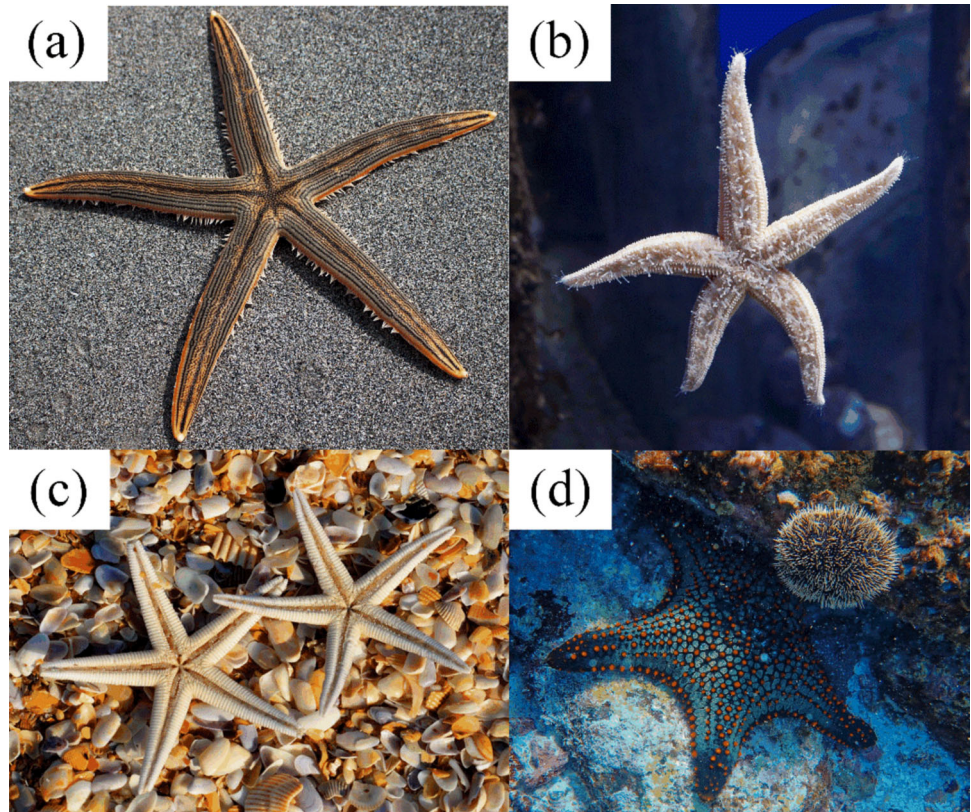
Body The eyes of a starfish are located at the end of the arms and enable them to sense light and darkness, water temperature, and environmental conditions. The mouth of a starfish is located at the center of its body, connected to the stomach by the esophagus. It has two stomachs: the cardiac stomach can be thrown up to the body to enclose and prey on food, while the pyloric stomach can digest the food.

Table 1 List of metaheuristic algorithms

Algorithm	Year	Algorithm	Year
Genetic algorithm [8]	1975	Dynamic differential annealed optimization [96]	2020
Simulated annealing [78]	1983	Equilibrium optimizer [83]	2020
Culture algorithm [111]	1994	Gradient-based optimizer [117]	2020
Particle swarm optimization [7]	1995	Group teaching optimization algorithm [118]	2020
Ant colony optimizer [22]	1996	Heap-based optimizer [119]	2020
Differential evolution [9]	1997	Lévy flight distribution [97]	2020
Harmony search [104]	2001	Marine predators algorithm [25]	2020
Big bang-big crunch [79]	2006	Manta ray foraging optimizer [41]	2020
Invasive weed optimization [84]	2006	Slime mould algorithm [42]	2020
Artificial bee colony [23]	2007	Tunicate swarm algorithm [43]	2020
Imperialist competitive algorithm [107]	2007	Aquila optimizer [44]	2021
Biogeography-based optimization [13]	2008	Atomic orbital search [94]	2021
Shuffled frog-leaping algorithm [27]	2006	Carnivorous plant algorithm [45]	2021
Firefly algorithm [28]	2008	Chameleon swarm optimization [46]	2021
Cuckoo search [29]	2009	Cooperation search algorithm [121]	2021
Gravitational search algorithm [80]	2009	Chaos game optimization [120]	2021
Charged system search [81]	2010	Horse herd optimization [48]	2021
Fireworks algorithm [85]	2010	Hunger game search [122]	2021
Brainstorming optimization [106]	2011	Lichtenberg algorithm [98]	2021
Teaching–learning-based optimization [15]	2011	Red fox optimization [49]	2021
Bat algorithm [30]	2012	Runge–Kutta method (RUN) [123]	2021
Flower pollination algorithm [31]	2012	Ali Baba and forty thieves [124]	2022
Krill herd [32]	2012	Alpine skiing optimization [125]	2022
Ray optimization [86]	2012	Artificial hummingbird algorithm [50]	2022
Water cycle algorithm [82]	2012	Artificial rabbits optimization [51]	2022
Colliding bodies optimization [87]	2014	Beluga whale optimization [26]	2022
Gray wolf optimizer [14]	2014	Cheetah optimizer [52]	2022
League championship algorithm [112]	2014	Dandelion optimizer [53]	2022
Soccer league competition algorithm [113]	2014	Dwarf mongoose optimization [54]	2022
Artificial algae algorithm [33]	2015	Gannet optimization algorithm [56]	2022
Moth flame optimization [34]	2015	Golden jackal optimization [55]	2022
Stochastic fractal search [88]	2015	Homonuclear molecules optimization [99]	2022
JAYA [108]	2016	Human felicity algorithm [126]	2022
Multi-verse optimizer [89]	2016	Honey badger algorithm [57]	2022
Sine cosine algorithm [90]	2016	Komodo Mlipir algorithm [58]	2022
Water evaporation optimization [91]	2016	Mountain gazelle optimizer [59]	2022
Whale optimization algorithm [35]	2016	Orca predation algorithm [60]	2022
Yin–Yang–pair optimization [110]	2016	Prairie optimization algorithm [61]	2022
Spotted hyena optimizer [36]	2017	Reptile search algorithm [62]	2022
Salp swarm algorithm [24]	2017	Seasons optimization algorithm [100]	2022
Thermal exchange optimization [92]	2017	Water flow optimizer [75]	2022
Socio-evolution and learning algorithm [114]	2018	White shark optimizer [65]	2022
Atom search optimization [93]	2019	Snake optimizer [63]	2022
Harris hawks optimization [16]	2019	Starling murmuration optimizer [64]	2022
Seagull optimization algorithm [37]	2019	Crayfish optimization algorithm [66]	2023
Sailfish optimizer [38]	2019	Fick’s law algorithm [102]	2023
Supply–demand-based optimization [115]	2019	RIME optimization algorithm [103]	2023
Artificial ecosystem optimization [95]	2020	Sand cat swarm optimization [67]	2023

Table 1 (continued)

Algorithm	Year	Algorithm	Year
Barnacle mating optimizer [74]	2020	Special relativity search [101]	2023
Bald eagle search [39]	2020	Termite life cycle optimizer [68]	2023
Chimp optimization algorithm [40]	2020	Young's double-slit optimizer [104]	2023

Fig. 2 Starfish; **a**, **b** body shape, **c** reproduction, **d** prey. <https://pexels.com>, <https://Pixabay.com>

Although starfish are marine creatures, they do not have fins, gills, or scales, nor can they swim like fish. Instead, they move via their tube feet in their arms and bodies. Starfish do not have a brain or heart, but their body is full of water, which simulates a circulatory system. Consequently, their arms move spontaneously and are not planned or controlled. Starfish typically have tough skin, small bumps, or thorns to protect themselves from predators, and can display a variety of colors, such as red and purple, which often blend with their surroundings and contribute to survival.

Reproduction Starfish are capable of both sexual and asexual reproduction [129]. In fact, it is often difficult to distinguish the sexuality of a starfish based on its appearance. During sexual reproduction, starfish release gametes

through openings on the central disk located between their arms. Males release sperm to fertilize the gametes in the marine environment, eventually resulting in the birth of baby starfish. A single starfish can produce over 60 million eggs, although very few can survive into adulthood and are vulnerable to being preyed upon by other creatures. In addition, when enough food is available, a starfish may engage in asexual reproduction, either via the autonomy of its arms or the fission of its central body. This process can generate multiple adults from a single larva, although it typically takes longer compared to sexual reproduction.

Prey and digestion Starfish are carnivorous [130], with consumption of small fish, snails, clams, and oysters, and occasionally prey on creatures larger than themselves. To catch their prey, the arms of a starfish close and warp

around it, and the tiny tube feet pry open the skin of their prey, such as clams or oysters. The stomach of a starfish then emerges from its mouth and outside its body to digest the creature or food. After digestion is complete, the stomach returns to the starfish's body. In starfish gathering, starfish may prey creatures cooperatively and share the information with each other.

Regeneration Starfish have a remarkable ability to regenerate [131]. When threatened by predators or caught by humans, the starfish can lose an arm to escape and subsequently regrow a new one. In fact, they can even regenerate an entirely new starfish from a lost arm and a small portion of their central body after several months or years. Regeneration can occur in escaping and accidental injury, which is controlled by a special type of connective tissue that responds to nervous signals for rapid softening.

3.2 Mathematical model of SFOA

The exploring, preying and regeneration behaviors of starfish can inspire us to develop a stochastic search optimization algorithm, named the starfish optimization algorithm (SFOA). Similar to other popular metaheuristic algorithms, SFOA has the exploration phase and exploitation phase, where the exploration phase simulates the exploring behaviors of starfish, and the exploitation phase is constructed by the preying and regeneration strategies. It should be noted that most popular metaheuristic algorithms utilize the vectorial search pattern in their exploration phases, which can provide good performance in handling the separable functions with searching in all dimensions. However, this pattern cannot ensure efficiency when handling non-separable functions [75]. The unidimensional search pattern can achieve better computational efficiency in dealing with non-separable functions, but it may be stuck into local convergence or have a slow convergence rate in optimization problems. To establish the mathematical models, a hybrid search pattern is developed in the exploration phase of SFOA by combining the five-dimensional and the unidimensional search patterns inspired by five arms in a starfish (the eyes are located in the arms). The proposed search pattern depends on the dimension D of the optimization problem, implementing the five-dimensional search pattern for $D > 5$ and unidimensional search pattern for $D \leq 5$, which can overcome shortcomings of slow convergence in the unidimensional search pattern and low efficiency in the vectorial search pattern. In the exploitation phase, SFOA is designed by the preying and generation strategies. The preying strategy in SFOA is the main updating process in the exploitation phase, utilizing the parallel two-directional search approach from the information of two starfish to urge the candidates to move toward better positions. Specifically, the regeneration phase

occurs only in the last starfish from the population ($i = N$), and this phase is very important for the global convergence capacity during the optimization process. Finally, SFOA has the same probability on the exploration phase and the exploitation phase in optimization. The details of mathematical models of SFOA are provided as follows.

3.2.1 Initialization

In the initialization phase of SFOA, the positions of starfish are generated randomly between the boundaries of design variables, which can be expressed as a matrix:

$$X = \begin{bmatrix} X_{11} & X_{12} & \cdots & X_{1D} \\ X_{21} & X_{22} & \cdots & X_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ X_{N1} & X_{N2} & \cdots & X_{ND} \end{bmatrix}_{N \times D} \quad (1)$$

where X is the matrix to save the positions of starfish with the size of $N \times D$, N is the population size, and D is the dimension of design variables. In the initialization phase, the position of each starfish in Eq. (1) is evaluated as:

$$X_{ij} = l_j + r(u_j - l_j), \quad i = 1, 2, \dots, N, \quad j = 1, 2, \dots, D \quad (2)$$

where X_{ij} means the j th dimension position of the i th starfish, r denotes a random number between (0, 1), and u_j and l_j are the upper bound and lower bound of design variables in the j th dimension, respectively. After generating the initialized matrix of positions, we can obtain the fitness values of all starfish by evaluating from the objective function, which can be also memorized as a vector:

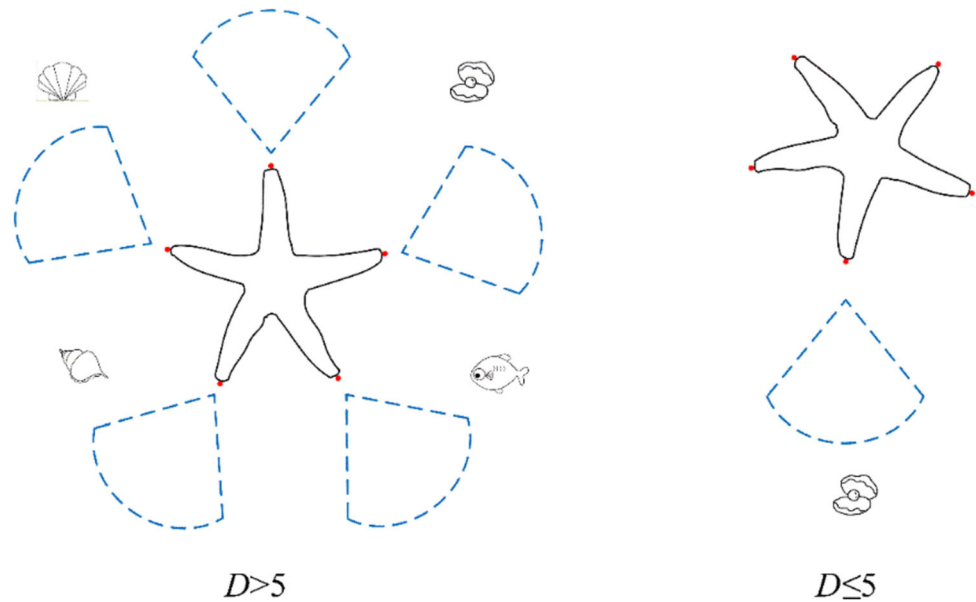
$$F = \begin{bmatrix} F(X_1) \\ F(X_2) \\ \vdots \\ F(X_N) \end{bmatrix}_{N \times 1} \quad (3)$$

where F is the matrix to store and update the obtained fitness values, with the size of $N \times 1$. After the initialization, SFOA enters the main loop, and starts to the exploration and exploitation phases.

3.2.2 Exploration phase

In order to model the explorative behavior of starfish, we establish the exploration phase in the SFOA algorithm, simulating the searching capacity of five arms of a starfish, where eyes are embedded at the end of arms, as shown in Fig. 3. In the exploration phase of SFOA, a novel search pattern is proposed, combining the five-dimensional search pattern for $D > 5$ and the unidimensional search pattern for $D \leq 5$ in different optimization problems. The threshold of

Fig. 3 Exploration phase of SFOA



dimension is considered based on the five arms (or eyes) in starfish.

If the dimension of the optimization problem is greater than 5 ($D > 5$), the search space of this problem is extensive, requiring starfish to move all five arms to explore the surrounding environment. In addition, the arms of starfish require knowledge of the best position among search agents to guide their movement. Therefore, we establish the mathematical model of this phase

$$\begin{cases} Y_{i,p}^T = X_{i,p}^T + a_1 (X_{\text{best},p}^T - X_{i,p}^T) \cos \theta, & r \leq 0.5 \\ Y_{i,p}^T = X_{i,p}^T - a_1 (X_{\text{best},p}^T - X_{i,p}^T) \sin \theta, & r > 0.5 \end{cases} \quad (4)$$

where $Y_{i,p}^T$ and $X_{i,p}^T$ denote the obtained and current positions of a starfish, respectively. $X_{\text{best},p}^T$ represents p -dimension of the current best position, p is five randomly selected dimensions among D dimensions, $r \in (0,1)$. a_1 and θ are calculated as:

$$a_1 = (2r - 1)\pi \quad (5)$$

$$\theta = \frac{\pi}{2} \cdot \frac{T}{T_{\max}} \quad (6)$$

where T is the current iteration, and $T_{\max}$ represents the maximum iteration. Sine and cosine terms denote that the arms of starfish may twist left or right to approach foods with the same probability. In the exploration phase, a_1 is randomly generated for the updating position in each candidate and iteration, and θ is changed with the number of iterations, $\theta \in [0, \pi/2]$, where two parameters can measure the effect of distance between the best position and the current position in the selected updating dimension. For the case of $D > 5$ in an optimization problem, the five-

dimensional search pattern is used in Eq. (4) to update only five dimensions of the positions to ensure the search capacity and improve the search efficiency compared with the vectorial search pattern. After that, when the updated position is outside the boundary of design variables, the arms have a tendency to remain in the previous position rather than moving to the updated position. The aforementioned can be mathematically expressed as:

$$X_{i,p}^{T+1} = \begin{cases} Y_{i,p}^T & l_{b,p} \leq Y_{i,p}^T \leq u_{b,p} \\ X_{i,p}^T & \text{otherwise} \end{cases} \quad (7)$$

where p denotes the updated dimension, $l_{b,p}$ and $u_{b,p}$ represent the bounds of design variables, respectively.

If the dimension of the optimization problem is not greater than 5 ($D \leq 5$), the exploration phase uses the unidimensional search pattern to update the position. In this case, only one arm in a starfish moves to search the food source, utilizing the location information from other starfish. The updated position can be established as:

$$Y_{i,q}^T = E_i X_{i,p}^T + A_1 (X_{k_1,p}^T - X_{i,p}^T) + A_2 (X_{k_2,p}^T - X_{i,p}^T) \quad (8)$$

where $X_{k_1,p}^T$ and $X_{k_2,p}^T$ are the p -dimensional positions from two randomly selected starfish, respectively, A_1 and A_2 are two random number between $(-1, 1)$, p is the randomly selected number in the D dimensions. E_i is the energy of the starfish, calculated as:

$$E_i = \frac{T_{\max} - T}{T_{\max}} \cos \theta \quad (9)$$

where θ is calculated by Eq. (6). Similar to the previous updating rule, if the obtained position of a starfish is out of

the boundary, the starfish tends to stay in the previous position rather than move to the updated position.

3.2.3 Exploitation phase

In SFOA, the preying and regeneration behaviors are considered in the exploitation phase to search for global solutions, so two updating strategies are designed in this phase. To model the preying phase of starfish, SFOA utilizes the parallel two-directional search strategy, which needs to utilize the information from other starfish, and the current best position of the population. We first calculate the five distances between the best location and other starfish, and then randomly select two distances as the acknowledgment to update the position for each starfish, utilizing the parallel two-directional search strategy. The distances can be calculated as:

$$d_m = (X_{\text{best}}^T - X_{m_p}^T), \quad m = 1, \dots, 5 \quad (10)$$

where d_m are five obtained distances between the global best and other starfish, while m_p are five randomly selected starfish. Thus, the updating rule of each starfish in the preying behavior is modeled as:

$$Y_i^T = X_i^T + r_1 d_{m1} + r_2 d_{m2} \quad (11)$$

where r_1 and r_2 are random number between (0,1), and d_{m1} and d_{m2} are random selected in d_m . The preying behavior of starfish is illustrated in Fig. 4. Based on the parallel two-directional search strategy, the candidates of starfish are moving toward the better guiding solution, while other candidates are moving backward in the same iteration. Thus, the candidates have the same capacity to overcome the local optima.

In addition, starfish have the vulnerability to other predators during their predation because of their slow movement. If a predator catches a starfish, the starfish may cut and lose an arm to evade capture. Thus, the regeneration phase of SFOA is implemented only in the last starfish among the population ($i = N$). Because the regeneration phase requires several months in life, the moving speed of the starfish is very slow. Thus, we model the updating rule in the regeneration phase, and the position is updated by:

$$Y_i^T = \exp(-T \times N / T_{\max}) X_i^T \quad (12)$$

where T is the current iteration, $T_{\max}$ means the maximum iterative number, and N is the population size. If the position obtained from Eq. (11) or Eq. (12) is out of the boundaries of design variables, we can set the position as:

$$X_i^{T+1} = \begin{cases} Y_i^T & l_b \leq Y_i^T \leq u_b \\ l_b & Y_i^T < l_b \\ u_b & Y_i^T > u_b \end{cases} \quad (13)$$

The regeneration phase can be described in Fig. 5. Although the regeneration phase has a very small number of function evaluations (only computing once time in the exploitation phase in each iteration) embedded in the exploitation phase, this phase is necessary to avoid local solutions and enhance global convergence.

3.3 The procedure of SFOA

As previously introduced, SFOA includes initialization, exploration, and exploitation phases. To implement SFOA, the exploration and exploitation phases are parallel to each other in the main loop, while the fitness values are calculated after updating positions from each phase. The stop criterion is determined by the maximum iteration. Once completed, SFOA outputs the global solution and the

Fig. 4 Preying behavior of SFOA

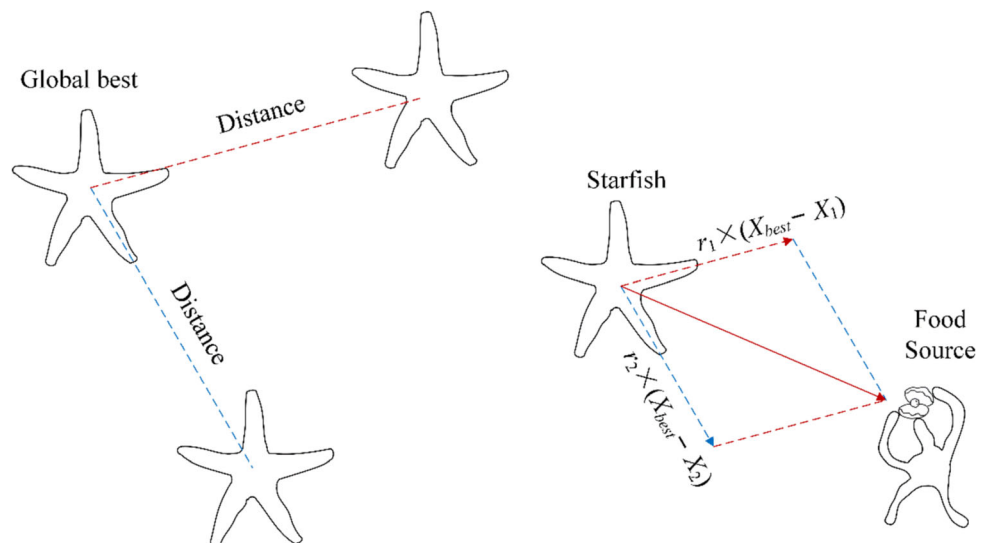


Fig. 5 Regeneration behavior of SFOA

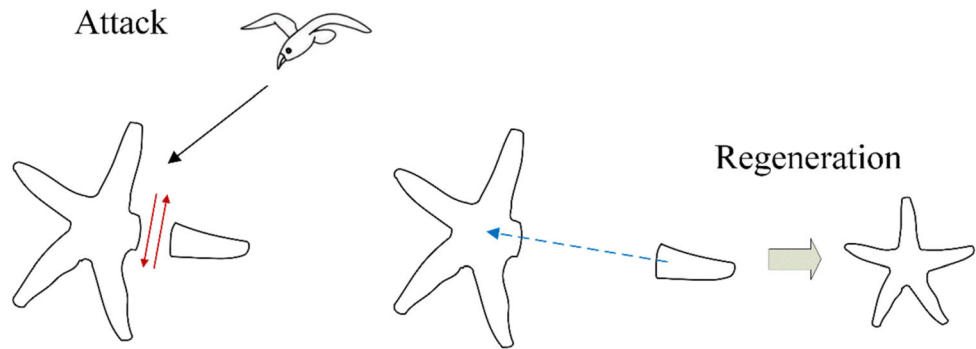
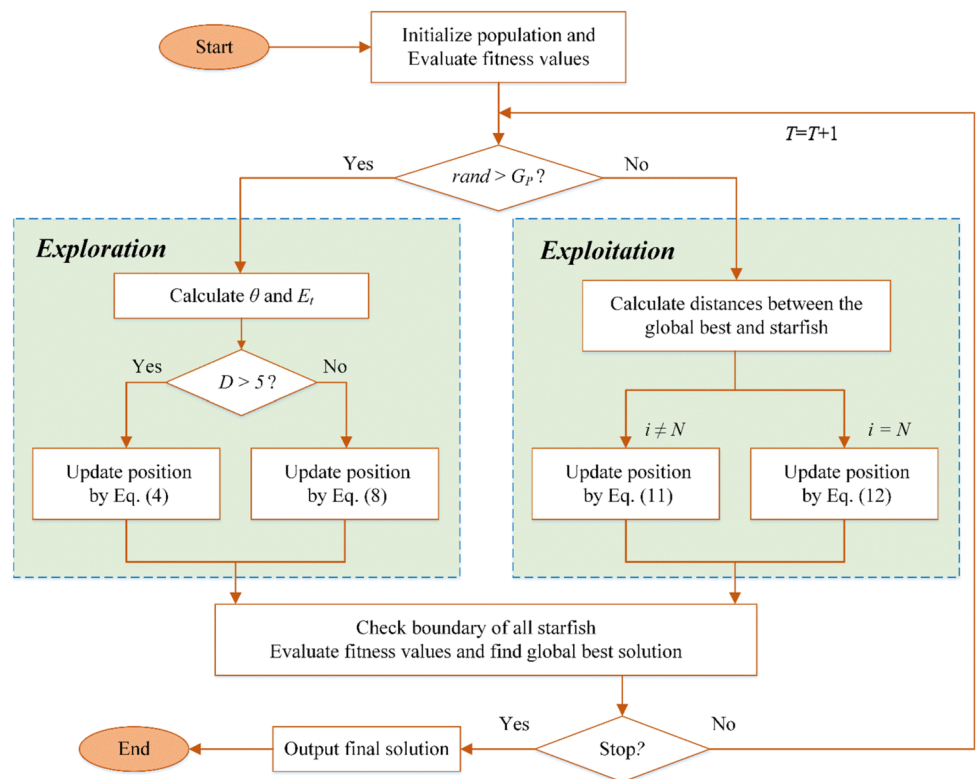


Fig. 6 Flowchart of SFOA



convergence curve. The detailed procedure of SFOA is as follows.

Step 1 Initialization phase. Input algorithmic parameters and problem information, and then generate the population randomly by Eq. (1) within the boundary of design variables, followed by the evaluation of fitness values. Then, SFOA enters the main loop after the initialization phase.

Step 2 In the main loop, the exploration and the exploitation phases have the same probability of implementation, depending on the comparison of a random number between (0, 1) and the algorithmic parameter G_p in SFOA, where G_p is equal to 0.5.

Step 3 Exploration phase. The hybrid search pattern is employed in this phase, using the five-dimensional and unidimensional search patterns. Two updating strategies

are designed for different dimensions. If the optimization problem has $D > 5$, the position is updated by Eq. (4), otherwise, updated by Eq. (8). After the updated positions are obtained, check the boundary of design variables using Eq. (7).

Step 4 Exploitation phase. The preying and regeneration phases are established. Using the parallel two-directional search pattern, the distances between the global solution and other starfish are calculated by Eq. (10), and then SFOA updates the position of each starfish using the preying behavior by Eq. (11). After that, the regeneration phase occurs when $i = N$, and its position is updated by Eq. (12). Once the positions are updated in the exploitation phase, check the boundary of design variables by Eq. (13).

Step 5 Termination. In SFOA, the exploration phase and exploitation phase are parallel to update positions of starfish, and the fitness values are evaluated after two phases. We find the current best solution of all starfish based on the obtained fitness values, then repeat the Steps 2–4 to update the positions of starfish and update the best solution in each iteration.

When the criterion of maximum iteration is satisfied, stop the main loop and output the final global solution.

To clearly show the procedure of SFOA, Fig. 6 provides the flowchart of SFOA, and the pseudo-code of SFOA is illustrated in Algorithm 1.

Algorithm 1 Pseudo-code of SFOA algorithm

Input: Algorithmic parameters (N , $T_{\max}$, and G_P)

Output: The global solution

```

1:   Initialize input parameters, problem information, boundary, etc.
2:   Initialize positions by Eq. (1) and evaluate the fitness values by Eq. (3)
3:   While  $T < T_{\max}$  Do
4:       If  $\text{rand} > G_P$       % Exploration phase
5:           Calculate  $\theta$  by Eq. (6) and energy  $E_t$  by Eq. (9)
6:           For each starfish Do
7:               If  $D > 5$  Do      % When the dimension is larger than 5
8:                   Generate five randomly  $p$ -dimensions among  $D$ 
9:                   For  $j = 1:5$  Do      % Update five randomly selected  $p$ -dimensions
10:                      Calculate  $a_1$  by Eq. (5)
11:                      Update the dimension of position in starfish using Eq. (4)
12:                  End For
13:                  Check the boundary of position by Eq. (7)
14:              Else      % When the dimension is not larger than 5
15:                  Generate two random numbers  $A_1$  and  $A_2$ 
16:                  Select a random dimension  $p$ 
17:                  Update the  $p$ -dimension of position using Eq. (8)
18:                  Check the boundary of starfish by Eq. (7)
19:              End If
20:          End For
21:       Else      % Exploitation phase
22:           Generate distances between the best position and starfish
23:           For each starfish Do
24:               Generate random number  $r_k$ , and update the position using Eq. (11)
25:               If  $i = N$  Do
26:                   Update the position using Eq. (12)      % regeneration phase
27:               End If
28:               Check the boundary of starfish by Eq. (13)
29:           End If      % Updating finished
30:       Calculate the fitness values for all starfish      % Obtain fitness values
31:       Obtain the global best solution in iteration  $T$  step
32:        $T = T + 1$ 
33:   End While
34:   Obtain global solution

```

3.4 Complexity of SFOA

The computational complexity of SFOA depends on the initialization, fitness evaluation, and updating of starfish. First, the computational complexity is $O(N \times D)$ at initialization, where N and D represent the population size and dimension, respectively. The exploration phase uses the hybrid search pattern for different dimension, which is calculated as $O(1/2 \times T_{\max} \times N \times 5)$ for $D > 5$ or $O(1/2 \times T_{\max} \times N \times 1)$ for $D \leq 5$, where $T_{\max}$ is the maximum iteration. In the exploitation phase, the complexity is calculated as $O(1/2 \times T_{\max} \times N \times D)$.

For an optimization problem with dimension $D > 5$, the total computational complexity of SFOA is calculated as:

$$\begin{aligned} O(\text{SFOA}) &= O(N \times D) + O(1/2 \times T_{\max} \times N \times 5) \\ &\quad + O(1/2 \times T_{\max} \times N \times D) \\ &= O\left(N \times T_{\max} \times D \times \left[\frac{1}{2} + \frac{5}{2D} + \frac{1}{T_{\max}}\right]\right) \\ &\approx O\left(N \times T_{\max} \times D \times \left[\frac{1}{2} + \frac{5}{2D}\right]\right) \end{aligned} \quad (14)$$

For $D \leq 5$ case, the total computational complexity of SFOA is calculated as:

$$\begin{aligned} O(\text{SFOA}) &= O(N \times D) + O(1/2 \times T_{\max} \times N \times 1) \\ &\quad + O(1/2 \times T_{\max} \times N \times D) \\ &= O\left(N \times T_{\max} \times D \times \left[\frac{1}{2} + \frac{1}{2D} + \frac{1}{T_{\max}}\right]\right) \\ &\approx O\left(N \times T_{\max} \times D \times \left[\frac{1}{2} + \frac{1}{2D}\right]\right) \end{aligned} \quad (15)$$

As comparison, a fast and popular algorithm PSO has the computational complexity $O(T_{\max} \times N \times D)$, which is larger than that of SFOA in most cases. Thus, the efficiency of SFOA is better than that of PSO. Furthermore, the number of fitness evaluations of SFOA is $N \times T_{\max}$, which is equal to that of PSO and some popular metaheuristic algorithms.

3.5 Theoretical analysis of SFOA and compared algorithms

Many metaheuristic algorithms inspired by various phenomena have been developed for optimization problems, and some popular or recently developed algorithms include PSO, ABC, GWO and MPA. Similar to other metaheuristic algorithms, SFOA is a population-based metaheuristic one that includes initialization, exploration and exploitation phases. However, SFOA diverges from other algorithms in that it draws inspiration from the behaviors of starfish, a

marine creature with a typical central body and five arms, including explorative, preying, and regeneration behavior. In contrast, PSO mimics the flock preying of birds or fishes, ABC simulates the on looking, scouting and employing behaviors of honey bees, GWO is inspired by the hierarchy and attacking strategies of gray wolves, and MPA mimics the preying behaviors in marine life with Brownian and Levy motions. In the proposed SFOA algorithm, the hybrid search pattern is proposed in the exploration phase by combining the five-dimensional with the unidimensional search patterns, mimicking the preying behaviors of starfish, to balance the search capacity and enhance the computational efficiency during the optimization process. Besides, SFOA utilizes the parallel two-directional search strategy to avoid local optima in the preying phase, and designs the regeneration phase to enhance global convergence in the optimization problem with the small number of fitness evaluations. SFOA has the same probability of implementing the exploration and exploitation phases, which is controlled by the algorithmic parameter $G_p = 0.5$. All the designed strategies ensure SFOA to search for the global solutions in solving optimization problems with satisfying accuracy and efficiency.

In addition, the mathematical model of SFOA is also different from other metaheuristic algorithms. PSO employs a single search strategy both with local search and global search. GWO uses the hierarchy to divide the population as different sub-division, and it implements several attacking strategies when preying from gray wolves. ABC has employed bees, onlooker bees, and scout bees, with different updating mechanisms for different bees. MPA consists of three updating rules depending on the high-velocity ratio, unit velocity ratio, and low-velocity ratio, where the eddy formation and memory mechanism are also introduced in MPA. When the proposed SFOA involves, the updating rules of exploration phase and exploitation phase in SFOA are different from those of other algorithms. First, the exploration phase has two updating rules for different dimensions ($D > 5$ and $D \leq 5$), when the search space is large ($D > 5$), five randomly selected dimensions of each starfish are updated, and when the search space is relatively small ($D \leq 5$), only one dimension of each starfish is updated, so the complexity of exploration behavior in SFOA is usually smaller than that of other metaheuristic algorithms. Second, the preying behavior and regeneration behavior are included in SFOA to update positions to control the local search capacity and ensure the global convergence, which can help SFOA to obtain global solutions during the optimization process. SFOA does not employ any additional strategies after exploration and exploitation phases, making it a simple and efficient algorithm with good accuracy and convergence capacity.

Due to the exploration behavior of SFOA, it is easy to be regarded as one of the variants of differential evolution (DE/rand-to-best/1). However, SFOA and the basic DE are very different. In the basic DE/rand-to-best/1, the best position and the randomly selected particle are used to explore, and the coefficient β is randomly generated between $[-1, 1]$. The crossover phase is implemented when the rand number is smaller than the probability of crossover probability. In SFOA, the coefficient θ is changed with the iteration and embedded into the sine and cosine functions. The coefficient a_1 is a random number equal to $(2r - 1)\pi$ in SFOA, larger than β in DE. Further, two strategies in SFOA's exploration phase are designed by the dimension of the optimization problem, much different from the basic DE. Further, the exploitation phase in SFOA includes both preying and regeneration behaviors, to enhance the exploitation capacity in optimization problems. Thus, the proposed SFOA is fundamentally different from the basic DE in the mathematical model. In the following section, the differences in experimental results between the proposed SFOA and DE are also illustrated.

4 Numerical experiment

To evaluate the performance of the proposed SFOA, we implement the numerical experiment with 65 benchmark functions, including 24 classical functions [14, 26], 29 functions from CEC 2017 test suite [70], and 12 functions from CEC 2022 test suite [132]. This section introduces the experimental details and 100 compared algorithms. Qualitative and quantitative analysis results are provided in the following subsections.

4.1 Experimental setup

The details of 24 classical benchmark functions are listed in Table 2, including function formulations, dimension (D), boundary of design variables, and reference minimum solution $f_{\min}$. Tables 3 and 4 provide the details about CEC 2017 and CEC 2022 test suites, respectively.

To deeply investigate the performance of SFOA, 100 different metaheuristic algorithms are used to compare with SFOA, including four algorithms before 2000, 13 algorithms between 2000 and 2009, 29 algorithms developed in 2010–2019, and 54 algorithms since 2020. The abbreviations of compared algorithms are as follows: AAA, ABC, AEO, AFT, AHA, AO, AOA, AOS, ASO, BA, BBBC, BBO, BES, BMO, BSO, BWO, CA, CBO, CCLNNA, CLHHEO, CLJAYA, CLPSO, CO, COA, COA2, CPA, CSA, CSS, DDAO, DDSRPAO, DE, DMO, DO, EESHHO, EJAYA, EO, EOOBLE, EPSO, FA, FDB, FPA, GA, GBO, GEO, GJO, GSA, GWO, HBA, HBO,

HGS, HHO, HPSOTVAC, HS, ICA, ICCO, ISGTOA, IWO, JAYA, KH, LFD, LSHADE, LSHADE-cnEpSin, LSHADE-SPACMA, MELGWO, MFO, MGO, MPA, MRFO, MSCPSO, MSGO, MVO, OPA, PDO, PO, PSO, QPSO, RIME, RO, RSA, RUN, SCA, SCSO, SFLA, SFS, SHO, SMA, SO, SOA, SQEOABC, SSA, STOA, TEO, TLBO, TLCO, TLSMA, TSA, WFO, WOA, WSO, and YDSE. Several state-of-the-art metaheuristic algorithms are also included, such as LSHADE-cnEpSin, LSHADE-SPACMA, FDB, MPA, EO, and WFO. Table 5 provides the basic information of compared algorithms. It should be noted that the population size N is set as 50, the maximum iteration $T_{\max}$ is set as 2000, and 30 independent runs are implemented for each algorithm. More details of compared algorithms such as algorithmic parameters and fitness evaluation numbers are provided in Supplementary Materials.

4.2 Qualitative analysis of SFOA

Figure 7 provides the qualitative analysis metrics of SFOA, including the shapes of tested functions, the search history, the trajectory, average fitness history, and convergence curve. For simplicity, we focus on ten classical benchmark functions (F1, F2, F5, F8, F12, F14, F16, F19, F21 and F23), including both unimodal and multimodal functions. In the second column of Fig. 7, the search histories show all the positions of starfish during the optimization process, illustrating that the positions of starfish are widely distributed in the entire search space, with more in the areas that promise the best solution during the exploration phase. The trajectory of the first dimension is also provided in the third column, which indicates the best starfish at any point during the optimization process. SFOA generally does not oscillate for unimodal functions, but it may oscillates in some multimodal functions (F12 and F21). The fourth and fifth columns show the average fitness values and convergence curves of SFOA, demonstrating its good convergence capability in solving unimodal and multimodal functions. In conclusion, the exploration and exploitation of SFOA are illustrated by the qualitatively analysis, particularly with reference to its updating mechanisms. SFOA achieves the balance between the exploration and exploitation, which can ensure the searching capacity and the convergence during the optimization.

4.3 Quantitative analysis of SFOA

This subsection illustrates the statistical results of SFOA in solving benchmark functions. The experimental results of SFOA with 30 independent runs are summarized in Table 6, where Best, Mean, Worst, and STD denote the minimum, average, maximum, and standard deviation for

Table 2 Details of 24 classical benchmark functions

Num	Name	Function	D	Boundary	$f_{\min}$
F1	Sphere	$F_1(x) = \sum_{i=1}^D x_i^2$	30	$[-100, 100]$	0
F2	Schwefel's 2.22	$F_2(x) = \sum_{i=1}^D x_i + \prod_{i=1}^D x_i $	30	$[-10, 10]$	0
F3	Powell Sum	$F_3(x) = \sum_{i=1}^D x_i ^{i+1}$	30	$[-1, 1]$	0
F4	Schwefel's 1.2	$F_4(x) = \sum_{i=1}^D \left(\sum_{j=1}^i x_j \right)^2$	30	$[-100, 100]$	0
F5	Schwefel's 2.21	$F_5(x) = \max_i \{ x_i , 1 \leq i \leq D\}$	30	$[-100, 100]$	0
F6	Rosenbrock	$F_6(x) = \sum_{i=1}^{D-1} [100(x_{i+1} - x_i^2) + (x_i - 1)^2]$	30	$[-30, 30]$	0
F7	Step	$F_7(x) = \sum_{i=1}^D (x_i + 0.5)^2$	30	$[-100, 100]$	0
F8	Quartic	$F_8(x) = \sum_{i=1}^D ix_i^4 + \text{random}[0, 1]$	30	$[-1.28, 1.28]$	0
F9	Zakharov	$F_9(x) = \sum_{i=1}^D x_i^2 + \left(\sum_{i=1}^D 0.5ix_i \right)^2 + \left(\sum_{i=1}^D 0.5ix_i \right)^4$	30	$[-5, 10]$	0
F10	Schwefel	$F_{10}(x) = -\sum_{i=1}^n (x_i \sin \sqrt{ x_i })$	30	$[-500, 500]$	$-418.98D$
F11	Periodic	$F_{11}(x) = 1 + \sum_{i=1}^D \sin^2(x_i) - \exp(\sum_{i=1}^D x_i^2)$	30	$[-10, 10]$	0
F12	Styblinski-Tang	$F_{12}(x) = 0.5 \sum_{i=1}^D (x_i^4 - 16x_i^2 + 5x_i)$	30	$[-5, 5]$	$-39.166D$
F13	Rastrigin	$F_{13}(x) = \sum_{i=1}^D [x_i^2 - 10 \cos(2\pi x_i) + 10]$	30	$[-5.12, 5.12]$	0
F14	Ackley 1	$F_{14}(x) = -20 \exp \left(-0.2 \sqrt{\left(\sum_{i=1}^D x_i^2 \right) / D} \right) - \exp \left(\left(\sum_{i=1}^D \cos(2\pi x_i) \right) / D \right) + 20 + e$	30	$[-32, 32]$	0
F15	Griewank	$F_{15}(x) = \sum_{i=1}^D x_i^2 / 4000 - \prod_{i=1}^D \cos(x_i / \sqrt{i}) + 1$	30	$[-600, 600]$	0
F16	Xin-She Yang N.4	$F_{16}(x) = \left(\sum_{i=1}^D \sin^2(x_i) - \exp(-\sum_{i=1}^D x_i^2) \right) \exp(-\sum_{i=1}^D \sin^2 \sqrt{ x_i })$	30	$[-10, 10]$	-1
F17	Penalized	$F_{17}(x) = \frac{\pi}{D} \left\{ 10 \sin(\pi y_1) + \sum_{i=1}^{D-1} (y_i - 1)^2 [1 + 10 \sin^2(\pi y_{i+1})] + (y_n - 1)^2 \right\} + \sum_{i=1}^D u(x_i, 10, 100, 4)$	30	$[-50, 50]$	0
F18	Penalized2	$F_{18}(x) = 0.1 \left\{ \sin^2(3\pi x_1) + \sum_{i=1}^D (x_i - 1)^2 [1 + \sin^2(3\pi x_i + 1)] + (x_D - 1)^2 [1 + \sin^2(2\pi x_D)] \right\} + \sum_{i=1}^D u(x_i, 5, 100, 4)$	30	$[-50, 50]$	0
F19	Foxholes	$F_{19}(x) = \left[\frac{1}{500} + \sum_{j=1}^{25} \frac{1}{j + \sum_{i=1}^2 (x_i - a_{ij})^6} \right]^{-1}$	2	± 65.536	0.998
F20	Kowalik	$F_{20}(x) = \sum_{i=1}^{11} \left a_i - \frac{x_i(b_i^2 + b_i x_2)}{b_i^2 + b_i x_3 + x_4} \right ^2$	4	$[-5, 5]$	0.000308
F21	Six Hump Camel	$F_{21}(x) = 4x_1^2 - 2.1x_1^4 + x_1^6/3 + x_1x_2 - 4x_2^2 + 4x_2^4$	2	$[-5, 5]$	-1.0316
F22	Shekel 5	$F_{22}(x) = -\sum_{i=1}^5 (x_i - a_i)(x_i - a_i)^T + c_i ^{-1}$	4	$[0, 10]$	-10.1532
F23	Shekel 7	$F_{23}(x) = -\sum_{i=1}^7 (x_i - a_i)(x_i - a_i)^T + c_i ^{-1}$	4	$[0, 10]$	-10.4028
F24	Shekel 10	$F_{24}(x) = -\sum_{i=1}^{10} (x_i - a_i)(x_i - a_i)^T + c_i ^{-1}$	4	$[0, 10]$	-10.5364

fitness values, respectively; the CPU time denotes the total CPU time(s) of 30 independent runs in each test function. According to the statistical results, SFOA achieves the global solutions in the majority of benchmark functions, such as F1–F5, F9, F11, F13–F16, F19, F21–F24 in classical functions, F1, F3, F4, and F9 in CEC 2017 test suite, F3, F5, F9, and F11 in CEC 2022 test suite, in which the obtained STD values are equal/close to 0 or far smaller than the mean values. Also, SFOA has high computational efficiency and has several functions with less than 10 s,

with 17 out of 24 test functions in classical functions, 23 out of 29 functions in CEC 2017 test suite, and 10 out of 12 functions in CEC 2022 test suite.

The basic SFOA includes one algorithmic parameter (G_P) to balance the exploration and exploitation phases, which is set as 0.5. In sensitivity analysis, we consider five different values of G_P (0, 0.25, 0.5, 0.75, 1.0), where $G_P = 0$ denotes that only the exploitation phase is used, and $G_P = 1.0$ means that only exploration phase is implemented. The statistical results of sensitivity analysis are

summarized in Table 7, which shows that both exploration and exploitation capacities are very important. When $G_p = 1.0$, SFOA failed to converge in F1–F5, F13–F16 functions. In functions F21–F24, the basic SFOA with $G_p = 0.5$ provides better performance than with other G_p values, illustrating the importance of balancing exploration and exploitation.

The scalability can verify the algorithm's performance in solving high-dimensional optimization problems. We considered four cases of different dimensions ($D = 100, 500, 1000$, and 5000) for classical functions F1–F18, where the parameters of SFOA are the same as the previous settings ($N = 50, T_{max} = 2000$, 30 independent runs). Table 8 summarizes the results of SFOA in solving high-dimensional optimization problems. SFOA still achieves the global solution or maintains good performance in functions F1–F5, F8, F9, F11, and F13–16 in scalability analysis, while providing worse results in functions F6, F7, F10,

F12, F17, and F18 with the increasing dimensions. As a comparison, several popular algorithms such as PSO, DE, BBO, and BBBC cannot achieve good solutions in the scalability analysis, which can be also seen in our previous study [26]. Thus, the proposed SFOA has a good capacity to solve some high-dimensional optimization problems.

4.4 SFOA versus 10 representative algorithms

In comparison, the statistical results of some classical and state-of-the-art metaheuristic algorithms are also provided, which can deeply investigate SFOA and compare algorithms in solving benchmark functions. Ten representative algorithms are compared with SFOA as follows.

Particle swarm optimization (075-PSO, 1995) achieves global convergence in four benchmark functions: F21 in classical functions, F3 and F9 in CEC 2017, F1 in CEC 2022, which is fewer than that of SFOA (totally 24

Table 3 Details of CEC 2017 benchmark functions

Type	Name	Function	f_{min}
Unimodal	CEC 2017-F1	Shifted and rotated Bent Cigar function	100
	CEC 2017-F3	Shifted and rotated Zakharov function	300
Multimodal	CEC 2017-F4	Shifted and rotated Rosenbrock's function	400
	CEC 2017-F5	Shifted and rotated Rastrigin's function	500
	CEC 2017-F6	Shifted and rotated expanded Schaffer's F6 function	600
	CEC 2017-F7	Shifted and rotated Lunacek Bi-Rastrigin function	700
	CEC 2017-F8	Shifted and rotated non-continuous Rastrigin's function	800
	CEC 2017-F9	Shifted and rotated Levy function	900
	CEC 2017-F10	Shifted and rotated Schwefel's function	1000
	CEC 2017-F11	Hybrid function 1 ($N = 3$)	1100
Hybrid	CEC 2017-F12	Hybrid function 2 ($N = 3$)	1200
	CEC 2017-F13	Hybrid function 3 ($N = 3$)	1300
	CEC 2017-F14	Hybrid function 4 ($N = 4$)	1400
	CEC 2017-F15	Hybrid function 5 ($N = 4$)	1500
	CEC 2017-F16	Hybrid function 6 ($N = 4$)	1600
	CEC 2017-F17	Hybrid function 7 ($N = 5$)	1700
	CEC 2017-F18	Hybrid function 8 ($N = 5$)	1800
	CEC 2017-F19	Hybrid function 9 ($N = 5$)	1900
	CEC 2017-F20	Hybrid function 10 ($N = 6$)	2000
Composite	CEC 2017-F21	Composition function 1 ($N = 3$)	2100
	CEC 2017-F22	Composition function 2 ($N = 3$)	2200
	CEC 2017-F23	Composition function 3 ($N = 4$)	2300
	CEC 2017-F24	Composition function 4 ($N = 4$)	2400
	CEC 2017-F25	Composition function 5 ($N = 5$)	2500
	CEC 2017-F26	Composition function 6 ($N = 5$)	2600
	CEC 2017-F27	Composition function 7 ($N = 6$)	2700
	CEC 2017-F28	Composition function 8 ($N = 6$)	2800
	CEC 2017-F29	Composition function 9 ($N = 3$)	2900
	CEC 2017-F30	Composition function 10 ($N = 3$)	3000

Range = $[-100, 100]$, $D = 10$

Table 4 Details of CEC 2022 benchmark functions

Name	Type	Function	$f_{\min}$
CEC 2022-F1	Unimodal	Shifted and full rotated Zakharov function	300
CEC 2022-F2	Basic functions	Shifted and rotated Rosenbrock's function	400
CEC 2022-F3		Shifted and rotated expanded Schaffer's F6 function	600
CEC 2022-F4		Shifted and rotated non-continuous Rastrigin's function	800
CEC 2022-F5		Shifted and rotated Levy function	900
CEC 2022-F6	Hybrid functions	Hybrid function 1 ($N=3$)	1800
CEC 2022-F7		Hybrid function 2 ($N=6$)	2000
CEC 2022-F8		Hybrid function 3 ($N=5$)	2200
CEC 2022-F9	Composite	Composition function 1 ($N=5$)	2300
CEC 2022-F10		Composition function 2 ($N=4$)	2400
CEC 2022-F11		Composition function 3 ($N=5$)	2600
CEC 2022-F12		Composition function 4 ($N=6$)	2700

Range = $[-100, 100]$, $D = 10$

functions achieved global convergence). The final rank of SFOA (67) is greater than that of PSO (125), and the total CPU time of SFOA (508 s) is also less than PSO (581 s). Thus, SFOA outperforms PSO in solving benchmark functions. (Pages 244–246 in Supplementary Materials).

Differential evolution (031-DE, 1997) achieves global convergence in six benchmark functions: F7 and F21 in classical functions, F6, F9, F26 in CEC 2017 test suite, F1 in CEC 2022 test suite, which is fewer than that of SFOA. The final rank (80) and CPU time (509 s) of SFOA are smaller than those of DE (rank: 113, CPU time: 1618 s). (Pages 112–114 in Supplementary Materials).

Teaching–learning-based optimization (093-TLBO, 2011) achieves global convergence in 11 benchmark functions: F1, F3, F7, F15, F19, F21–F24 in classical functions, F3 in CEC 2017 test suite, F1 in CEC 2022 test suite, which is fewer than that of SFOA. The final rank of SFOA (82) is greater than that of TLBO (104), and the total CPU time of SFOA (508 s) is also less than that of TLBO (1510 s). (Pages 298–300 in Supplementary Materials).

Gray wolf optimizer (047-GWO, 2014) achieves global convergence in two benchmark functions: classical functions F3 and F13, which is far fewer than that of SFOA. The final rank of SFOA (68) is greater than that of GWO (125), and the total CPU time of SFOA (508 s) is also less than that of GWO (765 s). (Pages 160–162 in Supplementary Materials).

LSHADE-cnEpSin (062. LSHADE-cnEpSin, 2017) achieves global convergence in ten benchmark functions: F19–F21, F24 in classical functions, F1, F3–F4 in CEC 2017 test suite, F1, F3, F5 in CEC 2022 test suite, which is fewer than that of SFOA. The final rank of SFOA (88) is greater than that of LSHADE-cnEpSin (101), and the total CPU time of SFOA (508 s) is also less than LSHADE-cnEpSin (2861 s). (Pages 205–207 in Supplementary Materials).

Harris hawks optimization (051-HHO, 2019) achieves global convergence in 13 benchmark functions: F1, F3–F4, F10–F16, F19–F21 in classical functions, none in CEC 2017 and CEC 2022 test suites, which is fewer than that of SFOA. Both the final rank and CPU time of SFOA (rank: 73, CPU time: 508 s) are better than those of HHO (rank: 113, CPU time: 1370 s). (Pages 172–174 in Supplementary Materials).

Equilibrium optimizer (036-EO, 2020) achieves global convergence in eight benchmark functions: F1, F3, F13, F15, F19, F21 in classical functions, F1 and F9 in CEC 2022 test suite, which is fewer than that of SFOA. Both the final rank and CPU time of SFOA (rank: 77, CPU time: 508 s) are better than those of EO (rank: 111, CPU time: 772 s). (Pages 127–129 in Supplementary Materials).

Marine predator algorithm (067-MPA, 2020) achieves global convergence in 16 benchmark functions: F15, F19–F24 in classical functions, F1, F3, F4, F6, F9 in CEC 2017 test suite, F1, F3, F9, F11 in CEC 2022 test suite. The final rank of SFOA (107) is worse than that of MPA (76), and the total CPU time of SFOA (508 s) is less than MPA (1351 s). (Pages 220–222 in Supplementary Materials).

Beluga whale optimization (016-BWO, 2022) achieves global convergence in 18 benchmark functions: F1–F7, F9–F19 in classical functions, none in CEC 2017 and CEC 2022 test suites, which is fewer than that of SFOA. Both the final rank and CPU time of SFOA (rank: 79, CPU time: 524 s) are better than those of BWO (rank: 105, CPU time: 973 s). (Pages 67–69 in Supplementary Materials).

Young's double-slit optimizer (100-YDSE, 2023) achieves global convergence in six benchmark functions: F19–F22, F24 in classical functions, F3 in CEC 2017 test suite, which is fewer than that of SFOA. The final rank of SFOA (88) is greater than that of YDSE (104), and the totally CPU time of SFOA (508 s) is less than that of YDSE (641 s). (Pages 319–321 in Supplementary Materials).

Table 5 List of compared algorithms

Abbreviation	Algorithm	Year	Abbreviation	Algorithm	Year
01. AAA	Artificial algae algorithm	2015	051. HHO	Harris hawks optimization	2019
02. ABC	Artificial bee colony	2007	052. HPSOTVAC	Hierarchical particle swarm optimizer with time-varying acceleration coefficients	2004
03. AEO	Artificial ecosystem optimization	2020	053. HS	Harmony search	2001
04. AFT	Ali baba and forty thieves	2022	054. ICA	Imperialist competitive algorithm	2007
05. AHA	Artificial hummingbird algorithm	2022	055. ICCO	Intelligent chaotic clonal optimizer	2002
06. AO	Aquila optimizer	2021	056. ISGTOA	Group teaching optimization algorithm with information sharing	2023
07. AOA	Arithmetic optimization algorithm	2021	057. IWO	Invasive weed optimization	2006
08. AOS	Atomic orbital search	2021	058. JAYA	JAYA	2016
09. ASO	Atomic orbital search	2019	059. KH	Krill Herd	2012
010. BA	Bat algorithm	2012	060. LFD	Lévy flight distribution	2020
011. BBBC	Big Bang-big crunch	2006	061. LSHADE	LSHADE	2014
012. BBO	Biogeography-based optimization	2008	062. LSHADE-cnEpSin	LSHADE-cnEpSin	2017
013. BES	Bald eagle search	2020	063. LSHADE-SPACMA	LSHADE-SPACMA	2017
014. BMO	Barnacles mating optimizer	2020	064. MELGWO	Gray wolf optimizer with memory, evolutionary operator and local search	2023
015. BSO	Brain storm optimization	2011	065. MFO	Month flame optimizer	2015
016. BWO	Beluga whale optimization	2022	066. MGO	Mountain Gazelle Optimizer	2022
017. CA	Cultural algorithm	1994	067. MPA	Marine predator algorithm	2020
018. CBO	Colliding bodies optimization	2014	068. MRFO	Manta ray foraging optimizer	2020
019. CCLNNA	Chaotic neural network with competitive learning	2021	069. MSCPSO	A multi-swarm self-adaptive and cooperative particle swarm optimization	2011
020. CLHHEO	Comprehensive learning Harris hawks-equilibrium optimization	2022	070. MSGO	Modified social group optimization	2020
021. CLJAYA	Comprehensive learning Jaya algorithm	2020	071. MVO	Multi-verse optimize	2016
022. CLPSO	Comprehensive learning particle swarm optimization	2006	072. OPA	Orca predation algorithm	2022
023. CO	Cheetah optimizer	2022	073. PDO	Prairie dog optimization	2022
024. COA	Chimp optimization algorithm	2020	074. PO	Political optimizer	2020
025. COA2	Crayfish optimization algorithm	2023	075. PSO	Particle swarm optimization	1995
026. CPA	Carnivorous plant algorithm	2021	076. QPSO	Quantum particle swarm optimization	2004
027. CSA	Cuckoo search algorithm	2009	077. RIME	Rime optimization algorithm	2023
028. CSS	Charged system search	2010	078. RO	Ray optimization	2012
029. DDAO	Dynamic differential annealed optimization	2020	079. RSA	Reptile search algorithm	2022
030. DDSRPSO	Directionally driven self-regulating particle swarm optimization	2016	080. RUN	Runge–Kutta optimizer	2021
031. DE	Differential evolution	1997	081. SCA	Sine–cosine algorithm	2016
032. DMO	Dwarf Mongoose Optimization	2022	082. SCSO	Sand cat swarm optimization	2023
033. DO	Dandelion optimizer	2022	083. SFLA	Shuffled frog-leaping algorithm	2006
034. EESHHO	Memetic Harris hawks optimization with elite evolutionary strategy	2021	084. SFS	Stochastic fractal search	2015
035. EJAYA	Enhanced Jaya	2021	085. SHO	Spotted hyena optimizer	2017
036. EO	Equilibrium optimizer	2020	086. SMA	Slime mould algorithm	2020
037. EOUBLE	Opposition-based learning equilibrium optimizer with Levy flight	2023	087. SO	Snake optimizer	2022
038. EPSO	Ensemble PSO	2017	088. SOA	Seagull optimization algorithm	2019
039. FA	Firefly algorithm	2008	089. SQEOABC		2023

Table 5 (continued)

Abbreviation	Algorithm	Year	Abbreviation	Algorithm	Year
				A self-adaptive quantum equilibrium optimizer with artificial bee colony	
040. FDB	Fitness distance balance	2020	090. SSA	Salp swarm algorithm	2017
041. FPA	Flower pollination algorithm	2020	091. STOA	Sooty tern optimization algorithm	2019
042. GA	Genetic algorithm	1975	092. TEO	Thermal exchange optimization	2017
043. GBO	Gradient-based optimizer	2020	093. TLBO	Teaching–learning-based optimization	2011
044. GEO	Golden eagle optimizer	2021	094. TLCO	Termite life cycle optimizer	2023
045. GJO	Golden jackal optimization	2022	095. TLSMA	Teaching–learning slime mould algorithm	2022
046. GSA	Gravitational search algorithm	2009	096. TSA	Tunicate swarm algorithm	2020
047. GWO	Gray wolf optimizer	2014	097. WFO	Water flow optimizer	2022
048. HBA	Honey badger algorithm	2022	098. WOA	Whale optimization algorithm	2016
049. HBO	Heap-based optimizer	2020	099. WSO	White shark optimizer	2022
050. HGS	Hunger game search	2021	100. YDSE	Young’s double-slit experiment optimizer	2023

The comparing results of SFOA and ten algorithms are summarized in Table 9. From the previous results among ten selected algorithms, we can see that SFOA outperforms nine compared algorithms in terms of accuracy (PSO, DE, TLBO, GWO, LSHADE-cnEpSin, HHO, EO, BWO, and YDSE), and is only worse than MPA. Besides, SFOA has better computational efficiency than ten compared algorithms.

Figure 8 provides the convergence curves of SFOA and several compared algorithms (PSO, DE, TLBO, EO, LSHADE-cnEpSin) in different benchmark functions. In classical functions F1, F5, F9, F14, F16 and F24, SFOA achieves global convergence and has the fastest convergence rate in these functions. As comparison, the convergence rates of TLBO and EO are slower than that of SFOA, while GWO, LSHADE-cnEpSin and PSO fail to converge in functions F1, F5 and F9, even LSHADE-cnEpSin oscillates before 200 steps in solving benchmark function F24. In CEC 2017 function F1 and F12, SFOA and LSHADE-cnEpSin both achieve global convergence and have faster convergence rate than other compared algorithms. In CEC 2017 functions F3, F9, F20 and F26, SFOA and compared algorithms have no significant difference in the convergence curves. In CEC 2022 functions F1 and F5, SFOA and compared algorithms have similar convergence curves, but SFOA outperforms other algorithms in CEC 2022 function F11. In all, SFOA can provide high-quality convergence behaviors in solving benchmark functions, both in classical functions and recent test suites (CEC 2017 and CEC 2022).

4.5 SFOA versus 100 optimizers

In this subsection, we discuss the results of SFOA comparing with 100 different optimizers, where the detailed

results are illustrated in Supplementary Materials. In this work, how to compare SFOA with 100 different metaheuristic algorithms is a difficult task in solving various benchmark functions. If the classical index such as the Friedman rank or Wilcoxon mean rank is implemented, the average performance of the proposed and compared algorithms will be flattened due to the large number of compared algorithms, and the good metaheuristic algorithms may not provide good solutions and convergences in specific optimization problems [133]. Therefore, we introduce the primitive one-on-one comparison scheme, in terms of the statistical mean and STD values from each benchmark function. This comparing scheme can directly display the difference between SFOA and any other compared algorithm, which is a robust and not affected by the number of compared algorithms. We also introduce the Rank 1 percentage index to measure the performance of SFOA and compared algorithms:

$$\theta_p = \sum_{k=1}^{N_A} \frac{R_1}{N_A}, \quad (16)$$

$$\theta_c = \sum_{k=1}^{N_A} \frac{R_2}{N_A} \quad (17)$$

where θ_p is the obtained Rank 1 percentage of SFOA, and θ_c is Rank 1 percentage of all compared algorithms, N_A is the number of compared algorithms ($N_A = 100$ in this work). R_1 and R_2 are the rank values in each algorithm. The values of Rank 1 percentage are calculated with the average and STD values from one-on-one comparison, and the full statistical results are provided in Supplementary Materials, Appendix B.

In the one-on-one comparison, SFOA is compared with each algorithm in each test function. If SFOA outperforms

the other algorithm in a test function, it ranks the first, and the other algorithm ranks the second. Conversely, if the compared algorithm outperforms SFOA, it ranks the first, and SFOA ranks the second. In cases where both SFOA and the compared algorithm achieve global solutions, both algorithms achieve the first rank simultaneously. Therefore, the percentage of times SFOA ranks first (Rank 1 percentage) can be obtained by calculating the sum of ranks achieved by SFOA compared to 100 optimizers. For example, among 100 compared algorithms, when SFOA outperforms 80 compared algorithms in a test problem, and no algorithm has the same solution with SFOA, the Rank 1 percentage of SFOA is calculated as $\theta_p = 80/100 = 80\%$ in this test function, while the Rank 1 percentage of the compared algorithm is $\theta_c = (100-80)/100 = 20\%$. More specifically, if SFOA and several compared algorithms both achieve global solution in the tested function and both rank the first, the Rank 1 percentage of SFOA is $\theta_p = 100/100 = 100\%$, while Rank 1 percentage of compared algorithm θ_c is dependent on the number of compared algorithms achieving global solutions. For instance, SFOA achieves the global solution in classical function F1, and its performance is not worse than any compared algorithm, so $\theta_p = 100\%$, while 37 compared algorithms achieve a global solution in F1, so $\theta_c = 37/100 = 37\%$. According to the Rank 1 percentage, the performances of SFOA and the compared algorithm in solving each benchmark function can be directly provided and to measure the ranking level of SFOA in solving different benchmark functions. Thus, the one-on-one comparison scheme can comprehensively investigate the pros and cons of SFOA in solving benchmark optimization problems.

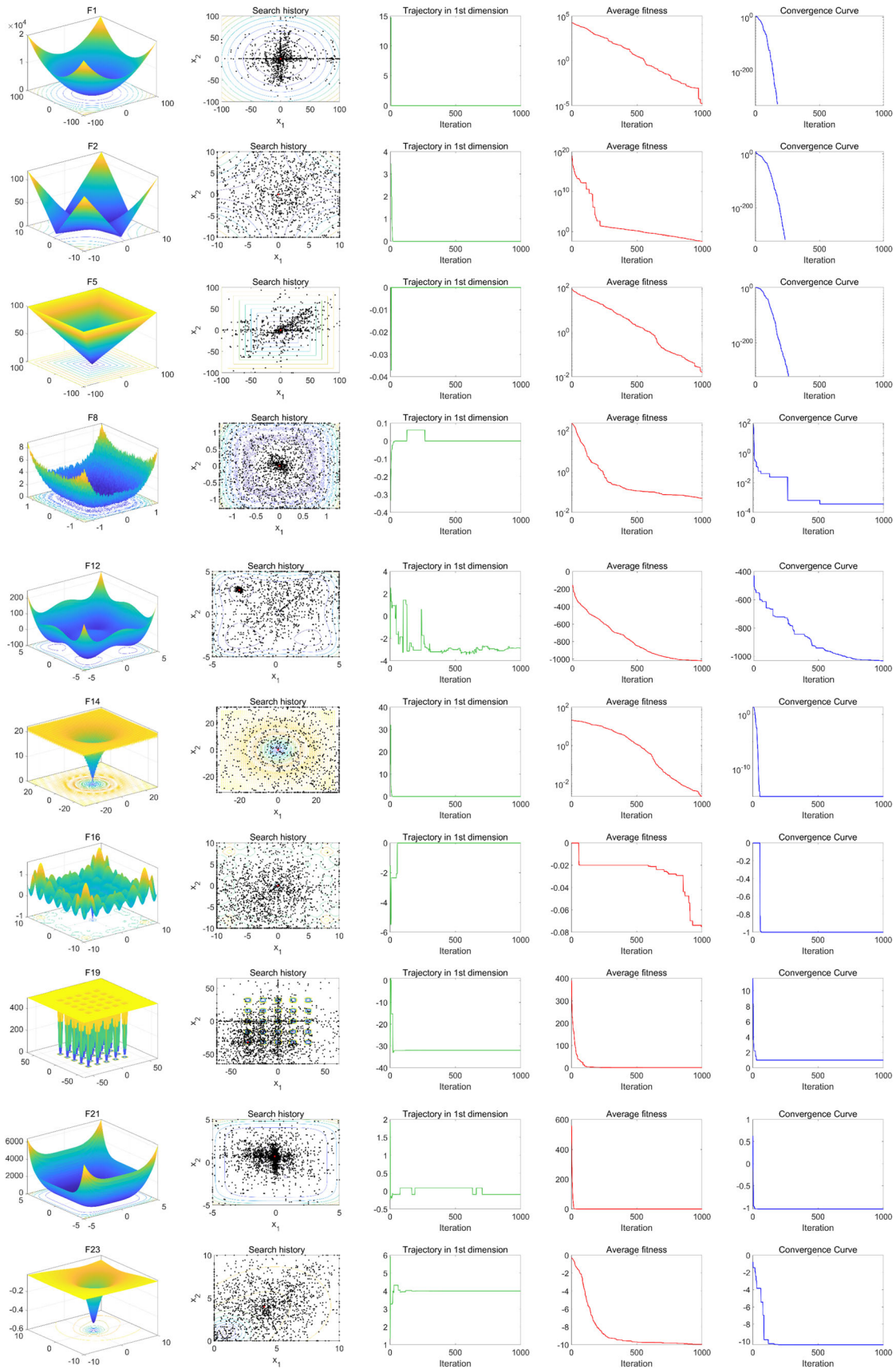
Table 10 summarizes the Rank 1 percentages of SFOA and 100 compared algorithms in benchmark functions. For 24 classical benchmark functions, SFOA achieves global solutions or ranks the first in all compared algorithms ($\theta_p = 100\%$) in 14 out of 24 functions (F1–F5, F9, F11, F13–F16, F22–F24), representing that SFOA is better or not worse than all compared algorithms in these functions. Meanwhile, SFOA achieves $\theta_p > 50\%$ in 23 out of 24 functions (F1–F6, F8–F24), having better accuracy than 50% to compared algorithms in solving these functions. The total percentage of Rank 1 in SFOA is 85.3% in classical functions, where 100 optimizers only have the percentage of Rank 1 as 32.8%. For CEC 2017 test functions, SFOA achieves $\theta_p > 90\%$ in 13 out of 29 functions (F1, F3–F4, F9, F12–F13, F15, F18–F19, F22, F25, F27 and F29), and achieves $\theta_p > 50\%$ in all CEC 2017 test functions. The total percentage of Rank 1 in SFOA from CEC 2017 test suite is 86.5%, which is greater than that of compared algorithms (15.2%). For CEC 2022 test suite, SFOA achieves $\theta_p = 100\%$ in two functions (F1 and F5), $\theta_p > 90\%$ in five functions (F1, F5, F6, F9 and F11), and

$\theta_p > 50\%$ in all CEC 2022 test functions. The total percentage of Rank 1 in SFOA from CEC 2022 test suite is 87.3%, which is greater than the obtained result of the compared algorithms (18.4%). Obviously, SFOA is superior in comparing algorithms on tested functions. It should be noted that according to the no free lunch theorem, SFOA cannot achieve global solutions in all functions. SFOA does not achieve good performance in F7–F8, F10, and F21 in classical functions with random terms or complex search space, also in CEC2017-F30 and CEC2022-F11 as composite functions.

Table 11 shows the winners of SFOA and compared algorithms. The winners depend on the total rank and CPU time of all tested functions, which can be seen in Supplementary Materials in Pages 322–324. Based on the results of solving benchmark functions, SFOA has 95 wins, two draws and three losses in terms of accuracy, which is only worse than three high-performance algorithms (LSHADE, MPA, and WFO); SFOA also has 97 wins and three losses in terms of efficiency (CPU time), only worse than AFT, HS, and WFO. Obviously, SFOA can achieve the top 5 in accuracy and top 3 in efficiency among 100 optimizers, illustrating that SFOA is a high-performance metaheuristic algorithm for solving different benchmark functions.

More specifically, the rankings of accuracy of LSHADE, MPA, and WFO are better than those of SFOA. However, SFOA has shorter CPU times (508 s) than that of LSHADE (1019 s) and MPA (1351 s), while the number of fitness evaluations from MPA (FEs = 200,000) is larger than that of SFOA (FEs = 100,000). Besides, the number of algorithmic parameters in SFOA (1 parameter) is smaller than that of LSHADE (4 parameters), MPA (2 parameters), and WFO (2 parameters), as shown in Supplementary Materials on Pages 12–14. Although the two compared algorithms, DMO and FDB, have the same ranking as SFOA, the CPU time of SFOA (508 s) is fewer than that of DMO (1851 s) and FDB (3940 s), and the numbers of fitness evaluations in DMO (FEs $\approx$ 287,000) and FDB (FEs = 400,000) are larger than that of SFOA. The rankings of efficiency of AFT and HS are better than those of SFOA, but SFOA has better accuracy than AFT and HS in solving benchmark functions. Thus, in 100 compared algorithms, only WFO outperforms SFOA both in accuracy and efficiency, with more algorithmic parameters than SFOA.

According to the previous experimental results, it is clear that SFOA is outstanding in solving benchmark functions compared with 100 different metaheuristic algorithms, which benefits from the ingenious mathematical models. In the exploration phase, the hybrid search pattern is used to combine with the five-dimensional and the unidimensional search patterns, only updating the positions of five dimension in the optimization problem



◀Fig. 7 Results of qualitative analysis by SFOA

with $D > 5$, and one dimension in the optimization problem with $D \leq 5$, which can greatly enhance the search efficiency in the non-separable functions comparing with the traditional vectorial search pattern by searching for the positions in all dimensions. Thus, the exploration phase can ensure SFOA performs well in solving complex optimization problems, such as CEC 2017 and CEC 2022 test suites. The hybrid search pattern strategy can satisfy the requirement for accuracy when solving most benchmark functions, which is more efficient than the traditional vectorial search pattern in popular metaheuristic algorithms. In the exploitation phase of SFOA, the preying phase employs the parallel two-directional search strategy to jump out of the local optima in multimodal functions and conceives the regeneration strategy to promote the global convergence and accelerate the convergence rate during the optimization process, which is dominating in the classical functions such as F1–F5 and F13–F16. Thus, the exploration phase for efficiency search and the exploitation phase for global convergence are complemented in SFOA, to achieve a good balance between exploration and exploitation. It should be mentioned that SFOA has the same number of function evaluations as PSO, and is better in computational efficiency than PSO (PSO is a very fast popular metaheuristic algorithm), due to its appropriate updating mechanism in the exploration and exploitation.

5 Engineering applications

To further investigate the performance of SFOA for real-world optimization problems, we implement ten engineering optimization problems, including cantilever beam design, welded beam design, spring design, pressure vessel design, three bar truss problem, speed reducer design, Himmelblau problem, multiple disk clutch brake design, car side impact design, and step cone pulley problem. As these optimization problems have different constraints, we use the penalty function method to handle the constraints due to its simplicity and efficiency. SFOA is compared with seven popular metaheuristic algorithms, including PSO, BBO, GWO, WOA, SSA, HHO, and AOA, with $N = 50$, $T_{\max} = 1000$, and other algorithmic parameters are the same as the previous values.

5.1 Cantilever beam design

The cantilever beam design problem is a popular real-world optimization problem, as shown in Fig. 9. The objective is to minimize the weight of the structure, under

the constraint on the vertical displacement of the free end, which has five design variables ($x_1 - x_5$). The mathematical model is expressed as [26, 104]:

$$\begin{aligned} \text{Min } f(\mathbf{x}) &= 0.0624(x_1 + x_2 + x_3 + x_4 + x_5) \\ \text{s.t. } g_1(\mathbf{x}) &= \frac{61}{x_2^3} + \frac{37}{x_2^2} + \frac{19}{x_3^3} + \frac{7}{x_4^3} + \frac{1}{x_5^3} - 1 \leq 0 \end{aligned}$$

where $0.01 \leq x_i \leq 100$ ($i = 1, 2, \dots, 5$).

In this engineering problem, SFOA is compared with seven metaheuristic algorithms, and the best solutions of all algorithms are summarized in Table 12, where the best value(s) are in boldface SFOA obtains the best solution with objective $f_{\min} = 13.03251$ and the design variable $\mathbf{x} = (5.97836, 4.87583, 4.46604, 3.47982, 2.13908)$, which has better performance than other compared algorithms.

5.2 Welded beam design

The second real-world optimization problem is the welded beam design, subjected to a vertical force at the end of the beam, as illustrated in Fig. 10, which is also usually investigated by researchers in the field of engineering optimization [6]. This problem has four design variables: $h = x_1$, $L = x_2$, $t = x_3$, $b = x_4$, where L denotes the length, h means the welding thickness, t is the height of the beam, and b represents the width of beam. The objective is to minimize the cost, with seven constraints of shear stress, bending stress, deflection, and bar buckling. The mathematical formulation of this problem is provided as:

$$\begin{aligned} \text{Find } \mathbf{x} &= [x_1, x_2, x_3, x_4] = [h, L, t, b] \\ \text{Min } f(\mathbf{x}) &= 1.10471x_1^2x_2 + 0.04811x_3x_4(14 + x_2) \\ \text{s.t. } g_1(\mathbf{x}) &= \tau(\mathbf{x}) - \tau_{\max} \leq 0, \\ g_2(\mathbf{x}) &= \sigma(\mathbf{x}) - \sigma_{\max} \leq 0, \\ g_3(\mathbf{x}) &= x_1 - x_4 \leq 0, \\ g_4(\mathbf{x}) &= 1.10471x_1^2 + 0.04811x_3x_4(14.0 + x_2) - 5.0 \leq 0, \\ g_5(\mathbf{x}) &= 0.125 - x_1 \leq 0, \\ g_6(\mathbf{x}) &= \delta(\mathbf{x}) - \delta_{\max} \leq 0, \\ g_7(\mathbf{x}) &= P - P_c(\mathbf{x}) \leq 0 \\ 0.1 \leq x_1 \leq 2, \quad & 0.1 \leq x_2 \leq 10, \quad 0.1 \leq x_3 \leq 10, \\ 0.1 \leq x_4 \leq 2, \end{aligned}$$

$$\begin{aligned} \text{where } \tau(\mathbf{x}) &= \sqrt{(\tau')^2 + 2\tau'\tau''\frac{x_2}{2R} + (\tau'')^2}, \quad \tau' = \frac{P}{\sqrt{2}x_1x_2}, \quad \tau'' \\ &= \frac{MR}{J}, \end{aligned}$$

$$\begin{aligned} M &= P\left(L + \frac{x_2}{2}\right), \quad R = \sqrt{\frac{x_2^2}{4} + \left(\frac{x_1 + x_3}{2}\right)^2}, \quad J \\ &= 2\left\{\sqrt{2}x_1x_2\left[\frac{x_2^2}{4} + \left(\frac{x_1 + x_3}{2}\right)^2\right]\right\}, \end{aligned}$$

Table 6 Statistical results of SFOA in solving benchmark functions

Classical	F1	F2	F3	F4	F5	F6	F7	F8
Best	0	0	0	0	0	2.05E + 01	2.27E-06	5.77E-05
Mean	0	0	0	0	0	2.16E + 01	2.13E-05	6.12E-04
Worst	0	0	0	0	0	2.32E + 01	8.71E-05	3.04E-03
STD	0	0	0	0	0	6.47E-01	1.95E-05	6.00E-04
CPU time/s	2.988	3.166	10.060	13.159	3.057	4.270	3.007	10.979
Classical	F9	F10	F11	F12	F13	F14	F15	F16
Best	0	- 10.834.50	0	- 1146.07	0	8.88E-16	0	- 1
Mean	0	- 8090.91	0	- 1088.66	0	8.88E-16	0	- 1
Worst	0	- 6069.75	0	- 1005.34	0	8.88E-16	0	- 1
STD	0	1125.21	0	38.81	0	0	0	0
CPU time/s	4.377	4.847	4.462	11.816	3.707	3.833	4.938	7.130
Classical	F17	F18	F19	F20	F21	F22	F23	F24
Best	9.13E-08	2.02E-06	0.9980	3.07E-04	- 1.0316	- 10.1532	- 10.4029	- 10.5364
Mean	6.39E-07	3.96E-04	0.9980	3.69E-04	- 1.0316	- 10.1532	- 10.4029	- 10.5364
Worst	4.10E-06	1.10E-02	0.9980	1.22E-03	- 1.0316	- 10.1532	- 10.4029	- 10.5364
STD	7.56E-07	2.01E-03	0	2.32E-04	1.74E-14	6.14E-15	4.66E-16	1.32E-15
CPU time/s	23.188	23.390	37.539	4.268	4.415	5.534	6.216	7.014
CEC 2017	F1	F3	F4	F5	F6	F7	F8	F9
Best	1.00E + 02	3.00E + 02	4.00E + 02	5.05E + 02	6.00E + 02	7.13E + 02	8.04E + 02	9.00E + 02
Mean	1.00E + 02	3.00E + 02	4.00E + 02	5.12E + 02	6.00E + 02	7.25E + 02	8.10E + 02	9.00E + 02
Worst	1.00E + 02	3.00E + 02	4.00E + 02	5.21E + 02	6.00E + 02	7.37E + 02	8.17E + 02	9.00E + 02
STD	2.45E-09	2.11E-14	2.12E-13	4.72E + 00	6.57E-05	5.85E + 00	3.94E + 00	3.66E-14
CPU time/s	4.306	4.176	4.307	5.150	7.539	5.278	5.185	5.439
CEC 2017	F10	F11	F12	F13	F14	F15	F16	F17
Best	1.16E + 03	1.10E + 03	1.20E + 03	1.30E + 03	1.40E + 03	1.50E + 03	1.60E + 03	1.70E + 03
Mean	1.46E + 03	1.10E + 03	1.49E + 03	1.31E + 03	1.41E + 03	1.50E + 03	1.60E + 03	1.73E + 03
Worst	2.00E + 03	1.11E + 03	1.92E + 03	1.31E + 03	1.42E + 03	1.50E + 03	1.61E + 03	1.74E + 03
STD	2.11E + 02	1.37E + 00	2.10E + 02	2.11E + 00	8.26E + 00	9.55E-01	2.42E + 00	7.70E + 00
CPU time/s	5.855	4.955	4.789	4.796	5.818	4.517	5.076	6.875

CEC 2017	F18	F19	F20	F21	F22	F23	F24	F25			
Best	1.80E + 03	1.90E + 03	2.00E + 03	2.20E + 03	2.20E + 03	2.60E + 03	2.50E + 03	2.60E + 03			
Mean	1.82E + 03	1.90E + 03	2.01E + 03	2.23E + 03	2.28E + 03	2.61E + 03	2.65E + 03	2.89E + 03			
Worst	1.97E + 03	1.91E + 03	2.03E + 03	2.32E + 03	2.30E + 03	2.62E + 03	2.75E + 03	2.95E + 03			
STD	3.05E + 01	9.88E-01	8.27E + 00	4.82E + 01	3.48E + 01	2.83E + 00	1.21E + 02	5.79E + 01			
CPU time/s	5.035	17.818	7.128	7.521	9.025	9.897	10.349	8.502			
CEC 2017	F26	F27	F28	F29	F30	CEC 2022-F1					
Best	2.80E + 03	3.09E + 03	3.10E + 03	3.13E + 03	3.39E + 03	3.00E + 02					
Mean	2.90E + 03	3.09E + 03	3.18E + 03	3.15E + 03	7.23E + 04	3.00E + 02					
Worst	2.98E + 03	3.09E + 03	3.41E + 03	3.17E + 03	1.25E + 06	3.00E + 02					
STD	3.81E + 01	5.27E-01	1.15E + 02	1.02E + 01	2.68E + 05	2.59E-14					
CPU time/s	11.111	11.519	10.241	9.319	20.486	3.809					
CEC 2022	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12
Best	4.00E + 02	6.00E + 02	8.04E + 02	9.00E + 02	1.80E + 03	2.00E + 03	2.20E + 03	2.53E + 03	2.50E + 03	2.60E + 03	2.86E + 03
Mean	4.04E + 02	6.00E + 02	8.12E + 02	9.00E + 02	1.80E + 03	2.01E + 03	2.21E + 03	2.53E + 03	2.50E + 03	2.60E + 03	2.86E + 03
Worst	4.09E + 02	6.00E + 02	8.23E + 02	9.00E + 02	1.80E + 03	2.02E + 03	2.22E + 03	2.53E + 03	2.50E + 03	2.60E + 03	2.86E + 03
STD	2.35E + 00	6.40E-05	5.06E + 00	4.72E-14	3.36E - 01	7.91E + 00	9.70E + 00	0	5.48E-02	1.11E-09	1.52E + 00
CPU time/s	3.700	6.871	4.736	4.934	3.543	7.420	8.699	7.421	6.901	10.019	10.230

Table 7 Sensitivity analysis of algorithmic parameter in SFOA

Para	$G_P = 0$		$G_P = 0.25$		$G_P = 0.5$		$G_P = 0.75$		$G_P = 1.0$	
	Aver	STD	Aver	STD	Aver	STD	Aver	STD	Aver	STD
F1	0	0	0	0	0	0	0	0	2.50E + 01	2.22E + 01
F2	0	0	0	0	0	0	0	0	4.45E-01	3.00E-01
F3	0	0	0	0	0	0	0	0	1.94E-12	6.48E-12
F4	0	0	0	0	0	0	0	0	8.19E + 02	5.63E + 02
F5	0	0	0	0	0	0	0	0	1.09E + 01	3.88E + 00
F6	2.86E + 01	1.12E-01	2.57E + 01	6.10E-01	2.72E + 01	4.87E-01	2.90E + 01	8.11E-02	2.14E + 03	2.96E + 03
F7	9.07E-01	4.62E-01	4.46E-03	2.87E-03	4.50E-02	2.97E-02	1.34E + 00	7.91E-01	1.86E + 01	1.32E + 01
F8	8.86E-04	7.45E-04	9.69E-04	1.08E-03	5.74E-04	5.96E-04	8.20E-04	6.99E-04	2.18E-02	1.46E-02
F9	0	0	0	0	0	0	0	0	8.47E + 00	5.84E + 00
F10	− 7994.97	859.09	− 7557.42	1102.62	− 6801.09	1022.24	− 6462.76	444.67	− 8438.95	662.05
F11	0	0	0	0	0	0	0	0	1.13E + 00	1.07E-01
F12	− 966.23	47.07	− 1020.53	42.51	− 1036.88	34.48	− 941.90	54.15	− 1077.34	25.28
F13	0	0	0	0	0	0	0	0	1.03E + 01	4.16E + 00
F14	8.88E-16	0	8.88E-16	0	8.88E-16	0	8.88E-16	0	1.43E + 00	6.70E-01
F15	0	0	0	0	0	0	0	0	1.13E + 00	3.08E-01
F16	− 1	0	− 1	0	− 1	0	− 1	0	2.66E-12	1.55E-12
F17	5.65E-02	4.03E-02	1.64E-04	1.33E-04	1.84E-03	1.05E-03	6.68E-02	4.54E-02	2.06E + 00	2.00E + 00
F18	9.75E-01	4.19E-01	3.16E-02	3.40E-02	7.96E-02	4.74E-02	1.00E + 00	4.92E-01	1.93E + 01	4.46E + 01
F19	9.98E-01	0.00E + 00	9.98E-01	9.22E-17	9.98E-01	1.70E-16	9.98E-01	3.95E-16	1.02E + 00	7.24E-02
F20	1.80E-03	5.06E-03	3.38E-04	1.67E-04	4.53E-04	3.33E-04	3.08E-04	1.41E-07	6.07E-04	9.43E-05
F21	− 1.0316	3.86E-12	− 1.0316	4.16E-14	− 1.0316	1.74E-14	− 1.0316	9.66E-10	− 1.0316	2.80E-05
F22	− 9.9848	0.9224	− 9.8164	1.2818	− 10.1532	6.14E-15	− 10.1532	1.54E-05	− 7.2011	1.1006
F23	− 10.2271	0.9629	− 10.4029	0.00E + 00	− 10.4029	4.66E-16	− 10.4029	5.41E-06	− 7.1383	1.5026
F24	− 10.5364	1.19E-15	− 10.5364	1.75E-15	− 10.5364	1.40E-15	− 10.5364	2.13E-05	− 7.5437	1.3895

$$\sigma(x) = \frac{6PL}{x_4 x_3^2}, \delta(x) = \frac{6PL^3}{Ex_3^2 x_4}, P_c(x) = 4.013E \sqrt{\frac{x_3^2 x_4^6}{36}} / L^2 \times \left(1 - \frac{x_3}{2L} \sqrt{\frac{E}{4G}}\right),$$

$$P = 6000 \text{ lb}, \quad L = 14 \text{ in.}, \quad E = 30 \times 10^6 \text{ psi}, \\ G = 12 \times 10^6 \text{ psi}, \quad \tau_{\max} = 13,600 \text{ psi}, \quad \sigma_{\max} = 30,000 \text{ psi}, \\ \delta_{\max} = 0.25 \text{ in.}$$

The results of the best solutions of SFOA and compared algorithms are listed in Table 13. SFOA obtains the global solution of $f_{\min} = 1.69526$ and $x = (0.20572, 3.25321, 9.03661, 0.20573)$. In this problem, SFOA and PSO both reach the minimum cost of the objective function, while other algorithms such as BBO, GWO, WOA, SSA, HHO, and AOA do not achieve the global optimum value.

5.3 Tension/compression spring design

The task of tension/compression spring design is to minimize the weight of the spring under the constraints of shear

stress, surge frequency, and deflection, with three design variables, the number of coils ($P = x_1$), the diameter of coil ($D = x_2$), and wire diameter ($d = x_3$), as shown in Fig. 11. The mathematical formulation is given as [50]:

$$\begin{aligned} \text{Min } f(x) &= (x_3 + 2)x_2 x_1^2 \neq \\ \text{s.t. } g_1(x) &= 1 - \frac{x_2^3 x_3}{71785 x_1^4} \leq 0, \\ g_2(x) &= \frac{4x_2^2 - x_1 x_2}{12566(x_2 x_1^3 - x_1^4)} + \frac{1}{5108 x_1^2} \leq 0, \\ g_3(x) &= 1 - \frac{140.45 x_1}{x_2^2 x_3} \leq 0, \\ g_4(x) &= \frac{x_1 + x_2}{1.5} - 1 \leq 0, \end{aligned}$$

$$\text{where } 0.05 \leq x_1 \leq 2.00, \quad 0.23 \leq x_2 \leq 1.30, \\ 2.00 \leq x_3 \leq 15.0.$$

Table 14 summarizes the obtained best solution of the tension/compression spring design problem solved by all algorithms. In this problem, SFOA can achieve the global solution of $f_{\min} = 0.01267$ and $x = (0.05170, 0.35686,$

Table 8 Results of SFOA in high-dimensional optimization problems (F1–F18)

Dim	D = 30			D = 100			D = 500			D = 1000			D = 5000		
	Fun	Aver	STD	Aver	STD	Aver	Aver	STD	Aver	STD	Aver	STD	Aver	STD	STD
F1		0	0	0	0	0	0	0	0	0	0	0	0	0	0
F2		0	0	0	0	0	0	0	0	0	0	0	0	0	0
F3		0	0	0	0	0	0	0	0	0	0	0	0	0	0
F4		0	0	0	0	0	0	0	0	0	0	0	0	0	0
F5		0	0	0	0	0	0	0	0	0	0	0	0	0	0
F6		2.73E + 01	3.99E-01	9.86E + 01	1.74E-01	4.99E + 02	1.23E-01	1.23E-01	9.99E + 02	1.19E-01	5.00E + 03	7.92E-02	5.00E + 03	7.92E-02	7.92E-02
F7		6.07E-02	3.82E-02	9.09E + 00	1.28E + 00	1.12E + 02	2.80E + 00	2.80E + 00	2.39E + 02	2.27E + 00	1.24E + 03	1.84E + 00	1.24E + 03	1.84E + 00	1.84E + 00
F8		1.08E-03	1.07E-03	8.97E-04	8.53E-04	6.86E-04	6.04E-04	6.04E-04	9.87E-04	1.07E-03	7.02E-04	7.66E-04	7.02E-04	7.66E-04	7.66E-04
F9		0	0	0	0	0	0	0	0	0	0	0	0	0	0
F10		− 7146.66	865.68	− 12,913.49	2343.74	− 28,573.76	7292.33	7292.33	− 41,324.32	7604.88	− 87,041.36	16,184.26	− 87,041.36	16,184.26	16,184.26
F11		0	0	0	0	0	0	0	0	0	0	0	0	0	0
F12		− 1043.81	34.09	− 2885.43	141.82	− 10,311.51	270.83	270.83	− 18,926.37	411.89	− 89,254.78	627.71	− 89,254.78	627.71	627.71
F13		0	0	0	0	0	0	0	0	0	0	0	0	0	0
F14		8.88E-16	0	8.88E-16	0	8.88E-16	0	0	8.88E-16	0	8.88E-16	0	8.88E-16	0	0
F15		0	0	0	0	0	0	0	0	0	0	0	0	0	0
F16		− 1	0	− 1	0	− 1	0	0	− 1	0	− 1	0	− 1	0	0
F17		2.02E-03	1.26E-03	1.63E-01	3.37E-02	8.69E-01	4.53E-02	4.53E-02	1.04E + 00	2.64E-02	1.16E + 00	4.76E-03	1.16E + 00	4.76E-03	4.76E-03
F18		1.09E-01	7.40E-02	9.47E + 00	1.04E + 00	5.00E + 01	4.78E-02	4.78E-02	1.00E + 02	8.60E-02	5.00E + 02	1.09E-02	5.00E + 02	1.09E-02	1.09E-02

Table 9 Comparison of SFOA and ten algorithms

Algorithms	Final rank value			CPU time/s		
	SFOA	Compared	Winner	SFOA	Compared	Winner
PSO	67	125	SFOA	508	581	SFOA
DE	80	113	SFOA	509	1618	SFOA
TLBO	82	104	SFOA	508	1510	SFOA
GWO	68	125	SFOA	508	765	SFOA
LSHADE-cnEpSin	88	101	SFOA	508	2861	SFOA
HHO	73	113	SFOA	508	1370	SFOA
EO	77	111	SFOA	508	772	SFOA
MPA	107	76	MPA	508	1351	SFOA
BWO	79	105	SFOA	524	973	SFOA
YDSE	88	104	SFOA	508	641	SFOA

11.28038). SFOA and PSO have better accuracy than other compared algorithms.

5.4 Pressure vessel design

In this engineering problem, the cylindrical pressure vessel design is investigated, as shown in Fig. 12. Four design variables are treated, shell thickness ($T_s = x_1$), head thickness ($T_h = x_2$), inner radius ($R = x_3$), and length of the cylinder ($L = x_4$), and the objective function is to minimize the construction weight, with four constraints on pressures and volumes. The mathematical formulation is shown as [64]:

$$\begin{aligned}
 \text{Min } f(\mathbf{x}) &= 0.6224x_1x_3x_4 + 1.7781x_2x_3^2 \\
 &\quad + 3.1661x_1^2x_4 + 19.84x_1^2x_3 \\
 \text{s.t. } g_1(\mathbf{x}) &= -x_1 + 0.0193x_3 \leq 0, \\
 g_2(\mathbf{x}) &= -x_2 + 0.00954x_3 \leq 0, \\
 g_3(\mathbf{x}) &= -\pi x_3^2x_4 - \frac{4}{3}\pi x_3^3 + 1296000 \leq 0, \\
 g_4(\mathbf{x}) &= x_4 - 240 \leq 0
 \end{aligned}$$

where $0 \leq x_1 \leq 99$, $0 \leq x_2 \leq 99$, $10 \leq x_3 \leq 200$, $10 \leq x_4 \leq 200$,

The obtained solutions of the pressure vessel design problem by all algorithms are given in Table 15. SFOA achieves the global solution with $f_{\min} = 5885.333$ and $\mathbf{x} = (0.77817, 0.38465, 40.31962, 200)$, ranking first in the compared algorithms. The obtained optimum values of PSO, BBO, and GWO are 6380.204, 6124.830, and 5907.560, which are larger than that of SFOA.

5.5 Three bar truss design

The three bar truss design problem is to minimize the cost of trusses under the static loads at the end of three bars, with two design variables of cross-sectional areas: $A_1 = x_1$, $A_2 = x_2$, as shown in Fig. 13. The mathematical formulation is expressed as follows [6]:

$$\begin{aligned}
 \text{Min } f(\mathbf{x}) &= (2\sqrt{2}x_1 + x_2) \times L \\
 \text{s.t. } g_1(\mathbf{x}) &= \frac{\sqrt{2}x_1 + x_2}{\sqrt{2}x_1^2 + 2x_1x_2}P - \sigma \leq 0, \\
 g_2(\mathbf{x}) &= \frac{x_2}{\sqrt{2}x_1^2 + 2x_1x_2}P - \sigma \leq 0, \\
 g_3(\mathbf{x}) &= \frac{1}{\sqrt{2}x_2 + x_1}P - \sigma \leq 0
 \end{aligned}$$

where $L = 100$ cm, $P = 2$ kN/cm², $\sigma = 2$ kN/cm², $0 \leq x_1$, $x_2 \leq 1$.

The global solution obtained by SFOA is $f_{\min} = 263.89584$ and $\mathbf{x} = (0.78868, 0.40825)$, which can be seen in Table 16. The results indicate that SFOA and PSO achieve the global solutions in the three bar truss design problem, outperforming other compared algorithms including BBO, GWO, WOA, SSA, HHO, and AOA.

5.6 Speed reducer design

Another popular engineering example is the speed reducer design [17], as shown in Fig. 14, with the objective function of the weight of the speed reducer. Seven design variables are considered: face width ($b = x_1$), middle of teeth ($m = x_2$), teeth in the pinion ($z = x_3$), lengths of the first and second shaft between bearings ($l_1 = x_4$, $l_2 = x_5$), diameters of the first and second shaft ($d_1 = x_6$, $d_2 = x_7$), while x_3 is the discrete design variable. Besides, 11 constraints are included in this problem, and the mathematical formulation is as follows:

$$\begin{aligned}
 \text{Find } \mathbf{x} &= [x_1, x_2, x_3, x_4, x_5, x_6, x_7] = [b, m, z, l_1, l_2, d_1, d_2] \\
 \text{Min } f(\mathbf{x}) &= 0.7854x_1x_2^2(3.3333x_3^2 \\
 &\quad + 14.9334x_3 - 43.0934) \\
 &\quad - 1.508x_1(x_6^2 + x_7^2)7.4777(x_6^3 + x_7^3) \\
 &\quad + 0.7854(x_4x_6^2 + x_5x_7^2) \\
 \text{s.t. } g_1(\mathbf{x}) &= 27/x_1x_2^2x_3 - 1 \leq 0, \\
 g_2(\mathbf{x}) &= 397.5/x_1x_2^2x_3^2 - 1 \leq 0,
 \end{aligned}$$

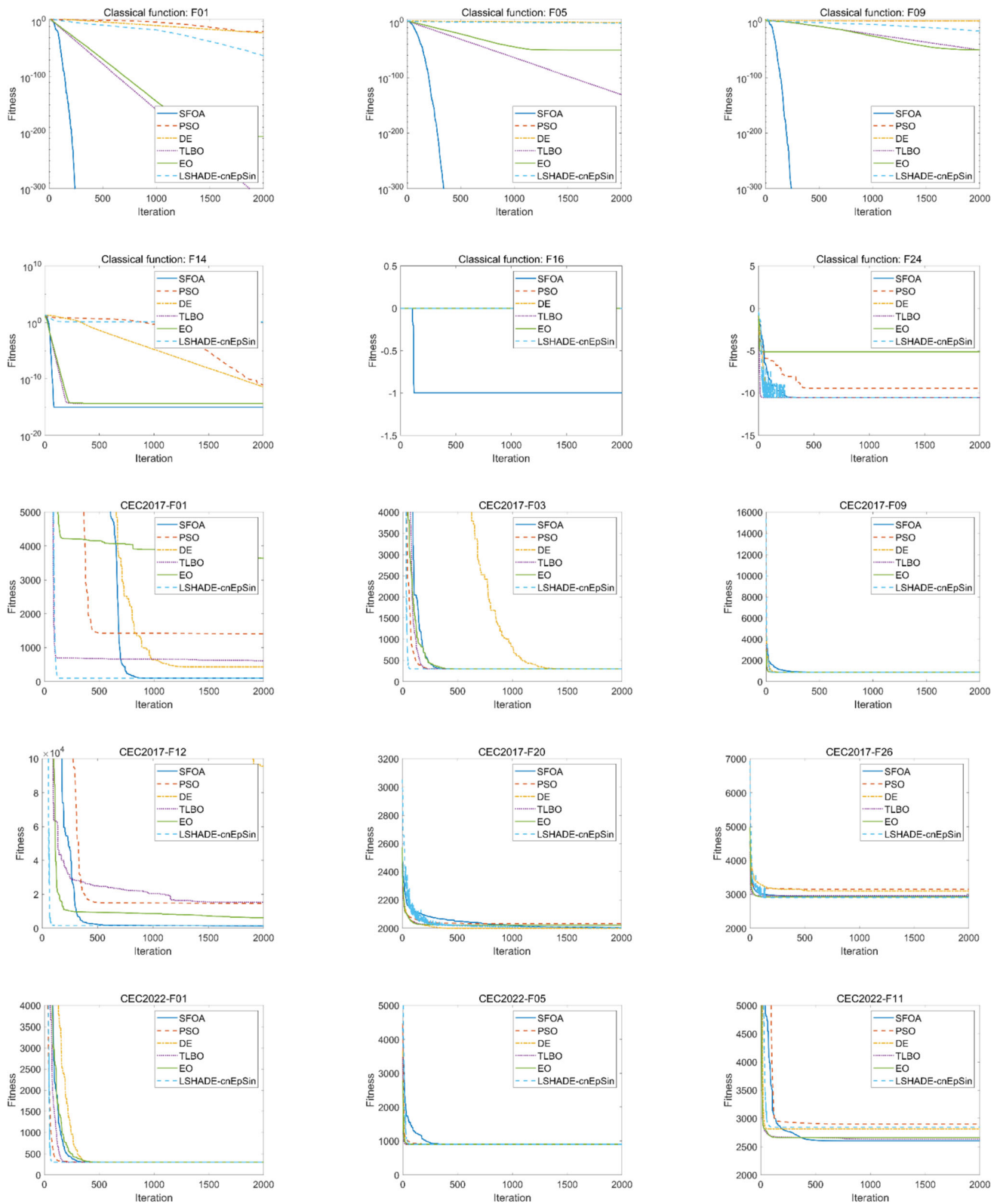


Fig. 8 Convergence curves of SFOA and several compared algorithms

Table 10 Rank 1 percentage in tested functions

Classical	Rank 1 percentage		CEC 2017	Rank 1 percentage		CEC 2022	Rank 1 percentage	
	SFOA (%)	Compared (%)		SFOA (%)	Compared (%)		SFOA (%)	Compared (%)
F1	100	37	F1	95	5	F1	100	31
F2	100	25	F3	100	28	F2	80	18
F3	100	39	F4	99	5	F3	84	17
F4	100	27	F5	76	24	F4	76	23
F5	100	18	F6	84	20	F5	100	11
F6	68	32	F7	70	30	F6	96	4
F7	47	52	F8	77	23	F7	79	21
F8	58	43	F9	99	13	F8	86	14
F9	100	23	F10	87	13	F9	97	30
F10	52	48	F11	86	14	F10	64	35
F11	100	26	F12	94	6	F11	99	1
F12	59	41	F13	97	3	F12	86	16
F13	100	35	F14	86	14			
F14	100	26	F15	97	3			
F15	100	37	F16	89	11			
F16	100	23	F17	81	19			
F17	71	29	F18	92	8			
F18	67	33	F19	96	4			
F19	95	38	F20	81	19			
F20	74	29	F21	68	32			
F21	55	60	F22	92	8			
F22	100	20	F23	83	19			
F23	100	21	F24	76	25			
F24	100	24	F25	92	8			
			F26	80	20			
			F27	90	10			
			F28	82	18			
			F29	92	8			
			F30	67	33			
Average	85.3	32.8	Average	86.5	15.2	Average	87.3	18.4

$$\begin{aligned}
g_3(\mathbf{x}) &= 1.93x_4^3/x_2x_3x_6^4 - 1 \leq 0, \\
g_4(\mathbf{x}) &= 1.93x_5^3/x_2x_3x_7^4 - 1 \leq 0, \\
g_5(\mathbf{x}) &= \sqrt{(745x_4/x_2x_3)^2 + 16.9 \times 10^6/110x_6^3} - 1 \leq 0, \\
g_6(\mathbf{x}) &= \sqrt{(745x_4/x_2x_3)^2 + 157.5 \times 10^6/85x_7^3} - 1 \leq 0, \\
g_7(\mathbf{x}) &= x_2x_3/40 - 1 \leq 0, \\
g_8(\mathbf{x}) &= 5x_2/x_1 - 1 \leq 0, \\
g_9(\mathbf{x}) &= x_1/12x_2 - 1 \leq 0, \\
g_{10}(\mathbf{x}) &= (1.56x_6 + 1.9)/x_4 - 1 \leq 0, \\
g_{11}(\mathbf{x}) &= (1.1x_7 + 1.9)/x_5 - 1 \leq 0
\end{aligned}$$

where $2.6 \leq x_1 \leq 3.6$, $0.7 \leq x_2 \leq 0.8$, $17 \leq x_3 \leq 28$, $7.3 \leq x_4 \leq 8.3$,

$$7.8 \leq x_5 \leq 8.3, 2.9 \leq x_6 \leq 3.9, 5.0 \leq x_7 \leq 5.5.$$

Table 17 displays the global solutions by each algorithm for the speed reducer design problem. SFOA achieves the global solution at $f_{\min} = 2996.348$ and $\mathbf{x} = (3.5, 0.7, 17, 7.3, 7.8, 3.35, 5.287)$. In comparison, the obtained solution by SFOA is better than those of PSO, BBO, GWO, WOA, WOA, SSA, and HHO.

5.7 Himmelblau problem

The Himmelblau problem was investigated in [6], with five design variables and several nonlinear constraints. The mathematical formulation is expressed as:

Table 11 Winners between SFOA and compared algorithms

Algorithm	Accuracy	Time	Algorithm	Accuracy	Time
01. AAA	SFOA	SFOA	051. HHO	SFOA	SFOA
02. ABC	SFOA	SFOA	052. HPSOTVAC	SFOA	SFOA
03. AEO	SFOA	SFOA	053. HS	SFOA	HS
04. AFT	SFOA	AFT	054. ICA	SFOA	SFOA
05. AHA	SFOA	SFOA	055. ICCO	SFOA	SFOA
06. AO	SFOA	SFOA	056. ISGTOA	SFOA	SFOA
07. AOA	SFOA	SFOA	057. IWO	SFOA	SFOA
08. AOS	SFOA	SFOA	058. JAYA	SFOA	SFOA
09. ASO	SFOA	SFOA	059. KH	SFOA	SFOA
010. BA	SFOA	SFOA	060. LFD	SFOA	SFOA
011. BBBC	SFOA	SFOA	061. LSHADE	LSHADE	SFOA
012. BBO	SFOA	SFOA	062. LSHADE-cnEpSin	SFOA	SFOA
013. BES	SFOA	SFOA	063. LSHADE-SPACMA	SFOA	SFOA
014. BMO	SFOA	SFOA	064. MELGWO	SFOA	SFOA
015. BSO	SFOA	SFOA	065. MFO	SFOA	SFOA
016. BWO	SFOA	SFOA	066. MGO	SFOA	SFOA
017. CA	SFOA	SFOA	067. MPA	MPA	SFOA
018. CBO	SFOA	SFOA	068. MRFO	SFOA	SFOA
019. CCLNNA	SFOA	SFOA	069. MSCPSO	SFOA	SFOA
020. CLHHEO	SFOA	SFOA	070. MSGO	SFOA	SFOA
021. CLJAYA	SFOA	SFOA	071. MVO	SFOA	SFOA
022. CLPSO	SFOA	SFOA	072. OPA	SFOA	SFOA
023. CO	SFOA	SFOA	073. PDO	SFOA	SFOA
024. COA	SFOA	SFOA	074. PO	SFOA	SFOA
025. COA2	SFOA	SFOA	075. PSO	SFOA	SFOA
026. CPA	SFOA	SFOA	076. QPSO	SFOA	SFOA
027. CSA	SFOA	SFOA	077. RIME	SFOA	SFOA
028. CSS	SFOA	SFOA	078. RO	SFOA	SFOA
029. DDAO	SFOA	SFOA	079. RSA	SFOA	SFOA
030. DDSRPSO	SFOA	SFOA	080. RUN	SFOA	SFOA
031. DE	SFOA	SFOA	081. SCA	SFOA	SFOA
032. DMO	SFOA/ DMO	SFOA	082. SCSO	SFOA	SFOA
033. DO	SFOA	SFOA	083. SFLA	SFOA	SFOA
034. EESHHO	SFOA	SFOA	084. SFS	SFOA	SFOA
035. EJAYA	SFOA	SFOA	085. SHO	SFOA	SFOA
036. EO	SFOA	SFOA	086. SMA	SFOA	SFOA
037. EOOBLE	SFOA	SFOA	087. SO	SFOA	SFOA
038. EPSO	SFOA	SFOA	088. SOA	SFOA	SFOA
039. FA	SFOA	SFOA	089. SQEOABC	SFOA	SFOA
040. FDB	SFOA/ FDB	SFOA	090. SSA	SFOA	SFOA
041. FPA	SFOA	SFOA	091. STOA	SFOA	SFOA
042. GA	SFOA	SFOA	092. TEO	SFOA	SFOA
043. GBO	SFOA	SFOA	093. TLBO	SFOA	SFOA
044. GEO	SFOA	SFOA	094. TLCO	SFOA	SFOA
045. GJO	SFOA	SFOA	095. TLSMA	SFOA	SFOA
046. GSA	SFOA	SFOA	096. TSA	SFOA	SFOA
047. GWO	SFOA	SFOA	097. WFO	WFO	WFO
048. HBA	SFOA	SFOA	098. WOA	SFOA	SFOA

Table 11 (continued)

Algorithm	Accuracy	Time	Algorithm	Accuracy	Time
049. HBO	SFOA	SFOA	099. WSO	SFOA	SFOA
050. HGS	SFOA	SFOA	100. YDSE	SFOA	SFOA

Accuracy—SFOA win/equal/lose: 95/2/3; CPU time—SFOA win/equal/lose: 97/0/3

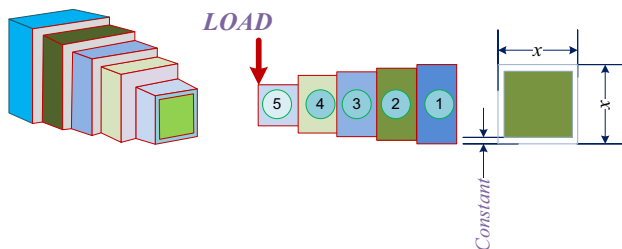
Min

$$f(\mathbf{x}) = 5.3578547x_3^2 + 0.8356891x_1x_5 + 37.293239x_1 - 40792.141$$

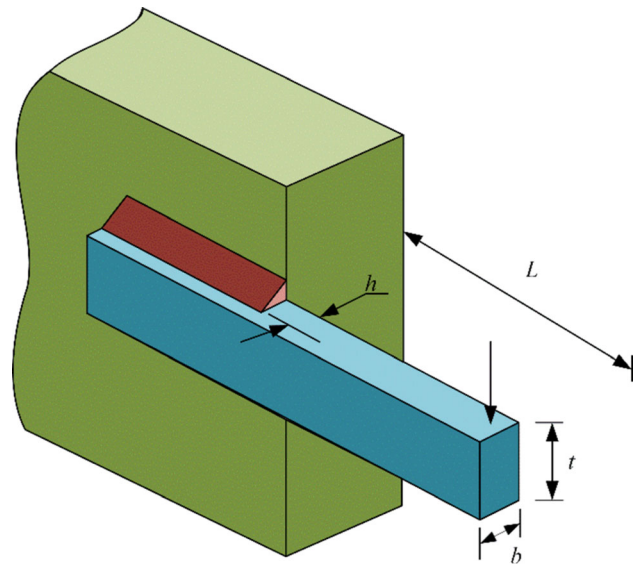
$$\begin{aligned} \text{s.t. } g_1(\mathbf{x}) &= 85.334407 + 0.0056858x_2x_5 \\ &\quad + 0.0006262x_1x_4 - 0.0022053x_3x_5 - 1 \leq 0, \\ g_2(\mathbf{x}) &= -85.334407 - 0.0056858x_2x_5 \\ &\quad - 0.0006262x_1x_4 + 0.0022053x_3x_5 \leq 0, \\ g_3(\mathbf{x}) &= 80.51249 + 0.0071317x_2x_5 \\ &\quad + 0.0029955x_1x_2 + 0.0021813x_3^2 - 110 \leq 0, \\ g_4(\mathbf{x}) &= -80.51249 - 0.0071317x_2x_5 \\ &\quad - 0.0029955x_1x_2 - 0.0021813x_3^2 + 90 \leq 0, \\ g_5(\mathbf{x}) &= 9.300961 + 0.0047026x_3x_5 \\ &\quad + 0.0012547x_1x_3 + 0.0019085x_3x_4 - 25 \leq 0, \\ g_6(\mathbf{x}) &= -9.300961 - 0.0047026x_3x_5 \\ &\quad - 0.0012547x_1x_3 - 0.0019085x_3x_4 + 20 \leq 0, \end{aligned}$$

where $78 \leq x_1 \leq 102$, $33 \leq x_2 \leq 45$, $27 \leq x_i \leq 45$ ($i = 3, 4, 5$).

The results of all algorithms for Himmelblau problem are provided in Table 18. SFOA achieves the best solution of $f_{\min} = -30,665.539$ and $\mathbf{x} = (78, 33, 29.995, 45, 36.776)$. According to the results, SFOA and PSO both

**Fig. 9** Cantilever beam**Table 12** Best solutions for cantilever beam

Algorithm	x_1	x_2	x_3	x_4	x_5	$f_{\min}$
SFOA	5.97836	4.87583	4.46604	3.47982	2.13908	13.03251
PSO	5.97723	4.87109	4.46757	3.48422	2.13904	13.03253
BBO	6.27566	4.73986	4.36115	3.88659	1.87646	13.15737
GWO	5.98138	4.86439	4.46793	3.47105	2.15479	13.03277
WOA	5.59558	5.51353	4.06182	4.61286	1.94506	13.52404
SSA	5.71773	6.05325	4.35153	3.29170	1.99952	13.32790
HHO	6.01528	5.13934	4.25415	3.39325	2.19244	13.06695
AOA	10.15803	4.40697	3.63385	3.34483	22.30126	27.28909

**Fig. 10** Welded beam**Table 13** Best solutions for welded beam design

Algorithm	x_1	x_2	x_3	x_4	$f_{\min}$
SFOA	0.20572	3.25321	9.03661	0.20573	1.69526
PSO	0.20573	3.25312	9.03662	0.20573	1.69526
BBO	0.38762	2.04817	6.55948	0.39046	2.31741
GWO	0.20434	3.28074	9.03588	0.20579	1.69728
WOA	0.34779	1.96759	8.04843	0.36440	2.51592
SSA	0.17274	4.03835	9.03244	0.20592	1.74724
HHO	0.12500	5.79827	8.95883	0.21044	1.89584
AOA	0.18722	5.59408	10.00000	0.21062	2.20208

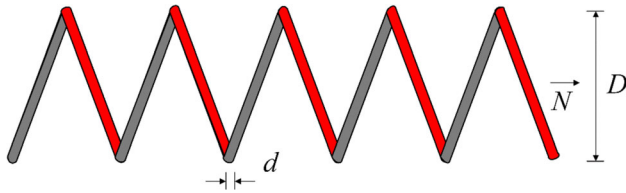


Fig. 11 Tension/compression spring

rank the first in this problem, having better optimum value than other compared algorithms.

5.8 Multiple disk clutch brake design

The multiple disk clutch brake design problem is depicted in Fig. 15, with the brake and discrete geotropic variables. Five continuous and discrete variables are included: inner radius ($r_i = x_1$), outer radius ($r_o = x_2$), thickness of the disk ($t = x_3$), actuating force ($F = x_4$), and number of friction surfaces ($Z = x_5$). The objective is to minimize the mass of the structure, under nine nonlinear constraints with radius, length, pressure, moment of bending, etc. The mathematical formulation is [50]:

$$\text{Find } \mathbf{x} = [x_1, x_2, x_3, x_4, x_5] = [r_i, r_o, t, F, Z]$$

$$\text{Min } f(\mathbf{x}) = \pi(r_o^2 - r_i^2)t(Z + 1)\rho$$

$$\begin{aligned} \text{s.t. } & g_1(\mathbf{x}) = r_o - r_i - \Delta r \geq 0, \\ & g_2(\mathbf{x}) = l_{\max} - (Z + 1)(t + \delta) \geq 0, \\ & g_3(\mathbf{x}) = p_{\max} - p_{rz} \geq 0, \\ & g_4(\mathbf{x}) = p_{\max}v_{sr\max} - p_{rz}v_{sr} \geq 0, \\ & g_5(\mathbf{x}) = v_{sr\max} - v_{sr} \geq 0, \\ & g_6(\mathbf{x}) = T_{\max} - T \geq 0, \\ & g_7(\mathbf{x}) = M_h - sM_s \geq 0, \\ & g_8(\mathbf{x}) = T \geq 0, \end{aligned}$$

$$\begin{aligned} M_h &= \frac{2}{3}\mu FZ \frac{r_o^3 - r_i^3}{r_o^2 - r_i^2}, \quad p_{rz} = \frac{F}{\pi(r_o^2 - r_i^2)}, \quad v_{sr} \\ &= \frac{2\pi n(r_o^3 - r_i^3)}{90(r_o^2 - r_i^2)}, \quad T = \frac{l_z \pi n}{30(M_h + M_f)} \end{aligned}$$

Table 14 Best solutions for tension/compression spring design

Algorithm	x_1	x_2	x_3	$f_{\min}$
SFOA	0.05170	0.35686	11.28038	0.01267
PSO	0.05210	0.36657	10.73376	0.01267
BBO	0.06218	0.66538	3.64333	0.01452
GWO	0.05000	0.31742	14.03232	0.01272
WOA	0.06422	0.73992	3.01467	0.01530
SSA	0.05000	0.31384	14.51404	0.01296
HHO	0.05695	0.49701	6.15078	0.01314
AOA	0.05000	0.31053	15.00000	0.01320

where $60 \leq x_1 \leq 80$, $90 \leq x_2 \leq 110$, $1 \leq x_3 \leq 3$, $600 \leq x_4 \leq 1000$, $2 \leq x_5 \leq 9$, $\Delta r = 20$ mm, $t_{\max} = 3$ mm, $t_{\min} = 1.5$ mm, $l_{\max} = 30$ mm, $\rho = 0.0000078$ kg/mm³, $Z_{\max} = 10$, $v_{sr\max} = 10$ m/s, $\mu = 0.5$, $s = 1.5$, $n = 250$ rpm, $M_s = 40$ N m, $M_f = 3$ N m, $p_{\max} = 1$ MPa, $l_z = 55$ kg mm², $T_{\max} = 15$ s, $F_{\max} = 1000$ N, $r_{i\min} = 55$ mm, $r_{o\max} = 110$ mm.

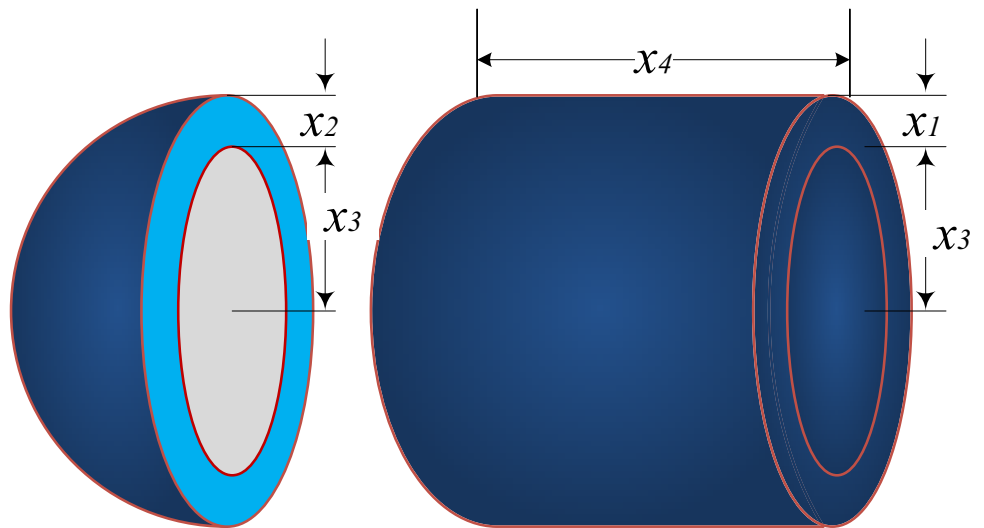
The obtained solutions by all algorithms are tabulated in Table 19. SFOA achieves the global solution at $f_{\min} = 30,159.289$ and $\mathbf{x} = (70, 90, 1, 1000, 2)$. In this case, four algorithms of SFOA, PSO, WOA, and HHO achieve the global solution, which outperform BBO, GWO, SSA, and AOA.

5.9 Car side impact design problem

This problem is a complex engineering optimization problem [6, 101], as depicted in Fig. 16, to optimize the vehicle side crashworthiness, with 11 design variables: thickness of B-pillar inner (x_1), B-pillar reinforcement (x_2), floor side inner (x_3), cross members (x_4), door beam (x_5), door beltline reinforcement (x_6), roof rail (x_7), materials of B-pillar inner (x_8), floor side inner (x_9), barrier height (x_{10}), and hitting position (x_{11}). Moreover, ten constraints are considered in this engineering problem, and the mathematical formulation based on the surrogate model is given as:

$$\begin{aligned} \text{Min } f(\mathbf{x}) &= 1.98 + 4.90x_1 + 6.67x_2 + 6.98x_3 \\ &\quad + 4.01x_4 + 1.78x_5 + 2.73x_7 \\ \text{s.t. } & g_1(\mathbf{x}) = 1.16 - 0.3717x_2x_4 - 0.00931x_2x_{10} \\ &\quad - 0.484x_3x_9 + 0.01343x_6x_{10} \leq 0 \\ & g_2(\mathbf{x}) = 46.36 - 9.9x_2 - 12.9x_1x_8 + 0.1107x_3x_{10} \leq 0 \\ & g_3(\mathbf{x}) = 33.86 + 2.95x_3 + 0.1792x_{10} \\ &\quad - 5.057x_1x_2 - 11x_2x_8 - 0.0215x_5x_{10} \\ &\quad - 9.98x_7x_8 + 22x_8x_9 - 32 \leq 0, \\ & g_4(\mathbf{x}) = 28.98 + 3.818x_3 - 4.2x_1x_2 + 0.0207x_5x_{10} \\ &\quad + 6.63x_6x_9 - 7.7x_7x_8 + 0.32x_9x_{10} - 4 \leq 0, \end{aligned}$$

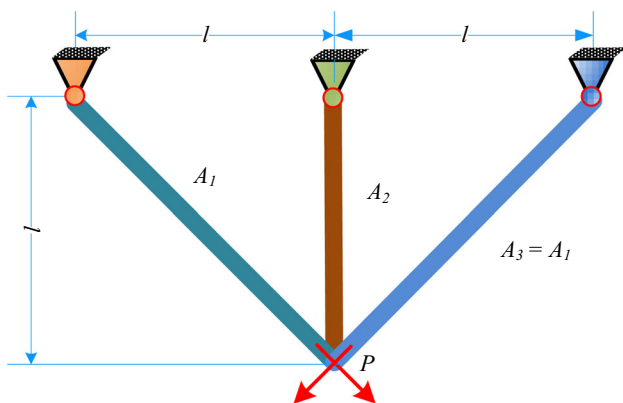
$$\begin{aligned} g_5(\mathbf{x}) &= 0.261 - 0.0159x_1x_2 - 0.188x_1x_8 - 0.019x_2x_7 \\ &\quad + 0.0144x_3x_5 + 0.0008757x_5x_{10} + 0.08045x_6x_9 \\ &\quad + 0.00139x_8x_{11} + 0.00001575x_{10}x_{11} - 0.32 \leq 0, \end{aligned}$$

Fig. 12 Cylindrical pressure vessel**Table 15** Best solutions for pressure vessel design

Algorithm	x_1	x_2	x_3	x_4	$f_{\min}$
SFOA	0.77817	0.38465	40.31962	200.00000	5885.333
PSO	0.99905	0.49383	51.76432	84.93603	6380.204
BBO	0.89606	0.44322	46.38590	129.88166	6124.830
GWO	0.78903	0.39062	40.87348	192.45125	5907.560
WOA	1.25891	0.62453	65.22523	10.00000	7336.516
SSA	0.87077	0.43042	45.11750	142.60277	6065.904
HHO	1.09769	0.53892	56.49103	53.94818	6696.613
AOA	1.46173	0.85818	74.17562	200.00000	26,389.828

Table 16 Best solutions for three bar truss design

Algorithm	x_1	x_2	$f_{\min}$
SFOA	0.78868	0.40825	263.89584
PSO	0.78863	0.40837	263.89584
BBO	0.80644	0.36022	264.11862
GWO	0.78794	0.41033	263.89665
WOA	0.79495	0.39078	263.92417
SSA	0.78711	0.41268	263.89765
HHO	0.78980	0.40508	263.89677
AOA	0.75106	0.53730	266.16151

**Fig. 13** Three bar truss

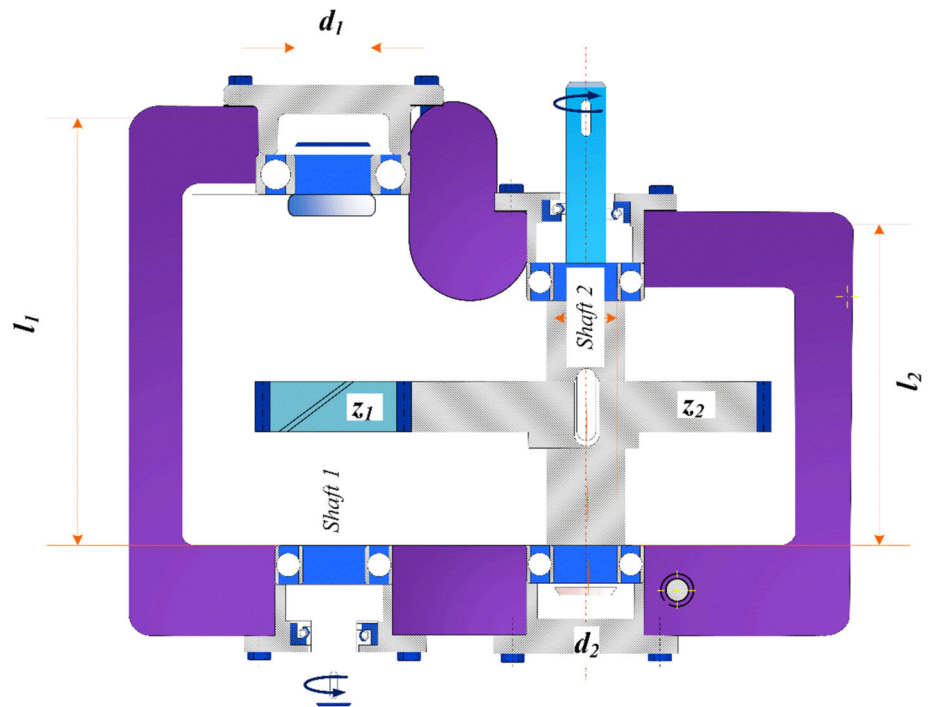
$$\begin{aligned}
 g_6(\mathbf{x}) = & 0.214 + 0.00817x_5 - 0.131x_1x_8 - 0.0704x_1x_9 \\
 & + 0.03099x_2x_6 - 0.018x_2x_7 + 0.0208x_3x_8 + 0.121x_3x_9 \\
 & - 0.00364x_5x_6 + 0.000715x_5x_{10} - 0.0005354x_6x_{10} \\
 & + 0.00121x_8x_{11} + 0.00184x_9x_{10} - 0.02x_2^2 - 0.32 \leq 0,
 \end{aligned}$$

$$\begin{aligned}
 g_7(\mathbf{x}) = & 0.74 - 0.61x_2 - 0.163x_3x_8 + 0.001232x_3x_{10} \\
 & - 0.166x_7x_9 + 0.227x_2^2 - 0.32 \leq 0,
 \end{aligned}$$

$$\begin{aligned}
 g_8(\mathbf{x}) = & 4.72 - 0.5x_4 - 0.19x_2x_3 - 0.0122x_4x_{10} \\
 & + 0.009325x_6x_{10} + 0.000191x_{11}^2 - 4 \leq 0,
 \end{aligned}$$

$$\begin{aligned}
 g_9(\mathbf{x}) = & 10.58 - 0.674x_1x_2 - 1.95x_2x_8 + 0.02054x_3x_{10} \\
 & - 0.0198x_4x_{10} + 0.028x_6x_{10} - 9.9 \leq 0,
 \end{aligned}$$

$$\begin{aligned}
 g_{10}(\mathbf{x}) = & 16.45 - 0.489x_3x_7 - 0.843x_5x_6 + 0.0432x_9x_{10} \\
 & - 0.0556x_9x_{11} - 0.000786x_{11}^2 - 15.7 \leq 0,
 \end{aligned}$$

Fig. 14 Speed reducer design**Table 17** Best solutions for speed reducer design

Algorithm	x_1	x_2	x_3	x_4	x_5	x_6	x_7	$f_{\min}$
SFOA	3.500	0.700	17.000	7.300	7.800	3.350	5.287	2996.348
PSO	3.600	0.700	17.000	7.300	8.300	3.350	5.287	3046.714
BBO	3.500	0.700	17.000	7.300	7.800	3.350	5.287	2996.359
GWO	3.502	0.700	17.000	7.367	7.821	3.353	5.287	2999.092
WOA	3.528	0.700	17.000	7.300	7.800	3.350	5.287	3007.235
SSA	3.578	0.700	17.000	7.904	8.270	3.477	5.287	3076.683
HHO	3.592	0.700	17.000	7.319	7.800	3.353	5.287	3033.450
AOA	3.600	0.700	17.000	7.300	8.300	3.558	5.316	3121.773

where $0.5 \leq x_1 \leq 1.5$, $0.45 \leq x_2 \leq 1.35$, $0.5 \leq x_3 \leq 1.5$, $0.5 \leq x_4 \leq 1.5$, $0.875 \leq x_5 \leq 2.625$, $0.4 \leq x_6 \leq 1.2$, $0.4 \leq x_7 \leq 1.2$, $0.192 \leq x_8 \leq 0.345$, $0.192 \leq x_9 \leq 0.345$, $0.5 \leq x_{10} \leq 1.5$, $0.5 \leq x_{11} \leq 1.5$.

Table 20 summarizes the obtained results by the compared algorithms in solving the car side impact design problem. SFOA achieves the global solution at $f_{\min} = 23.5616$ and $\mathbf{x} = (0.5, 1.234, 0.5, 1.187, 0.875, 0.892, 0.4, 0.345, 0.192, 1.5, 0.572)$. From the results, we can see that the obtained optimum value $f_{\min}$ by SFOA is better than those of PSO, BBO, GWO, WOA, SSA, HHO and AOA, showing that SFOA has good performance of exploration and exploitation in solving this complex engineering optimization problem.

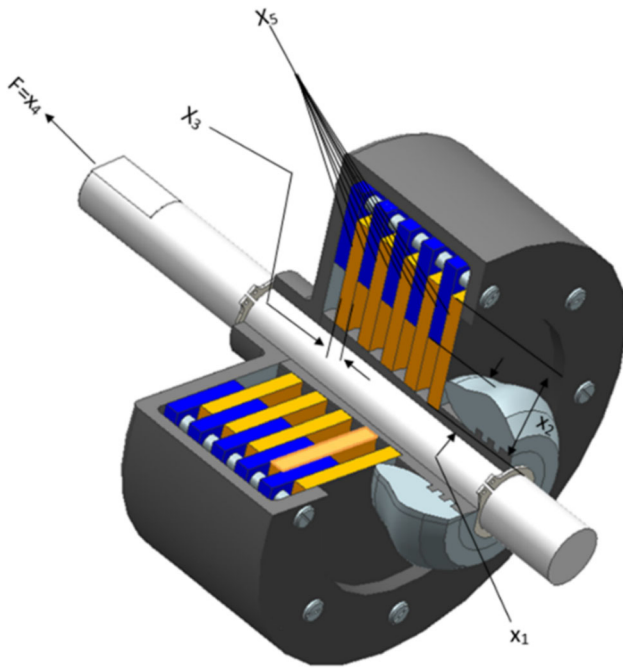
5.9.1 Step cone pulley problem

The step cone pulley problem is the last engineering optimization problem, as shown in Fig. 17, while the objective is to minimize the weight of step cone pulley, with five design variables and 11 nonlinear equality and inequality constraints. The mathematical formulation is given as follows [1, 15]:

$$\begin{aligned}
 &\text{Find } \mathbf{x} = [x_1, x_2, x_3, x_4, x_5] = [d_1, d_2, d_3, d_4, \omega] \\
 &\text{Min } f(\mathbf{x}) = \rho w \left[d_1^2 \left\{ 1 + \left(\frac{N_1}{N} \right)^2 \right\} + d_2^2 \left\{ 1 + \left(\frac{N_2}{N} \right)^2 \right\} \right. \\
 &\quad \left. + d_3^2 \left\{ 1 + \left(\frac{N_3}{N} \right)^2 \right\} + d_4^2 \left\{ 1 + \left(\frac{N_4}{N} \right)^2 \right\} \right]
 \end{aligned}$$

Table 18 Best solutions for Himmelblau problem

Algorithm	x_1	x_2	x_3	x_4	x_5	$f_{\min}$
SFOA	78.000	33.000	29.995	45.000	36.776	– 30,665.539
PSO	78.000	33.000	29.995	45.000	36.776	– 30,665.539
BBO	78.000	33.259	30.154	44.997	36.379	– 30,640.374
GWO	78.000	33.008	30.008	45.000	36.749	– 30,663.222
WOA	78.000	33.000	30.088	38.762	39.075	– 30,485.933
SSA	78.000	33.035	30.016	43.665	37.265	– 30,626.964
HHO	78.000	33.000	30.043	44.807	36.733	– 30,652.925
AOA	102.000	33.000	30.734	45.000	29.390	– 29,422.198

**Fig. 15** Multiple disk clutch brake

$$\begin{aligned}
 \text{s.t. } & h_1(\mathbf{x}) = C_1 - C_2 = 0, \\
 & h_2(\mathbf{x}) = C_1 - C_3 = 0, \\
 & h_3(\mathbf{x}) = C_1 - C_4 = 0, \\
 & g_{1,2,3,4}(\mathbf{x}) = -R_i \leq 2, \\
 & g_{5,6,7,8}(\mathbf{x}) = (0.75 \times 745.6998) - P_i \leq 0
 \end{aligned}$$

$$\begin{aligned}
 \text{where } & C_i = \frac{\pi d_i}{2} \left(1 + \frac{N_i}{N} \right) + \frac{(N_i/N - 1)^2}{4a} + 2a, \quad i = 1, 2, 3, 4 \\
 & R_i = \exp(\mu \{ \pi - 2 \sin^{-1} \{ (N_i/N - 1) \times d_i/2a \} \}), \quad i = 1, 2, 3, 4 \\
 & P_i = st\omega(1 - R_i) \frac{\pi d_i N_i}{60}, \quad i = 1, 2, 3, 4
 \end{aligned}$$

$$t = 8 \text{ mm}, \quad s = 1.75 \text{ MPa}, \quad \mu = 0.35, \quad \rho = 7200 \text{ kg/m}^3, \quad a = 3 \text{ mm}.$$

The obtained optimum values by the algorithms for the step cone pulley problem are tabulated in Table 21. The global best solutions achieved by SFOA are $f_{\min} = 16.0903$ and $\mathbf{x} = (38.4140, 52.8586, 70.4727, 84.4957, 90)$. According to the results, SFOA has the better optimum value than seven compared algorithms, indicating the

effectiveness of SFOA for solving constrained optimization problems.

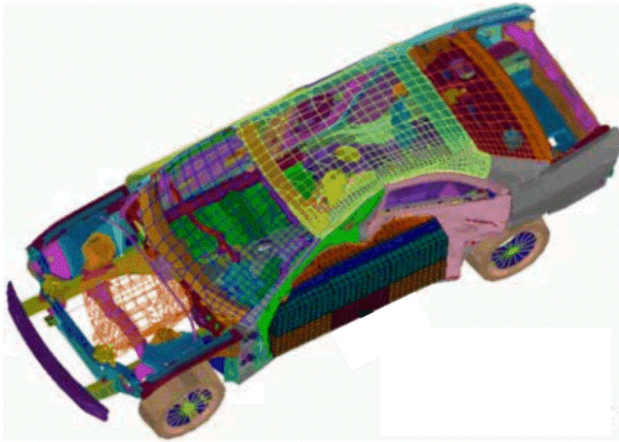
5.9.2 Statistical results of engineering problems

Based on the previous studies, SFOA obtains the global best solution in solving ten real-world engineering optimization problems, which outperforms other metaheuristic algorithms and provides good performance when considering different constraints. To further verify the robustness of SFOA for real-world optimization problems, each algorithm runs 30 times independently, and SFOA is also compared with seven popular metaheuristic algorithms: PSO, BBO, GWO, WOA, SSA, HHO, and AOA. The statistical results are summarized in Table 22, including four metrics: minimum value (min), mean value (mean), maximum value (max), and standard deviation (STD), while the best values are in boldface.

Results indicate that SFOA achieves very stable solutions in all ten engineering optimization problems, while the STD values are far smaller than the mean values. In comparison, PSO, WOA, and AOA fail to obtain stable solutions in the step cone pulley problem, while BBO, SSA, and HHO also provide unstable solutions in welded beam design problem, GWO cannot get global convergence in the pressure vessel design and the step cone pulley problem. Moreover, the obtained mean and STD values of the compared algorithms are worse than those of SFOA, except for PSO in Himmelblau problem, and PSO and HHO in multiple disk clutch brake design problem. Finally, SFOA achieves good and stable solutions in solving real-world optimization problems, demonstrating its effectiveness for engineering optimization. Based on these results, SFOA has the capacity to solve constrained optimization problems with nonlinear objective functions and multiple constraints, consisting of up to 11 design variables (car side design problem). It should be mentioned that the tested engineering problems are related to mechanical design. For more complex optimization or high-dimensional real-world optimization problems in

Table 19 Best solutions for multiple disk clutch brake design

Algorithm	x_1	x_2	x_3	x_4	x_5	$f_{\min}$
SFOA	70.000	90.000	1.000	1000.000	2.000	30,159.289
PSO	70.000	90.000	1.000	774.913	2.000	30,159.289
BBO	69.999	90.000	1.000	831.530	2.000	30,160.024
GWO	69.999	90.000	1.000	600.000	2.000	30,160.756
WOA	70.000	90.000	1.000	600.000	2.000	30,159.289
SSA	70.000	90.000	1.000	957.134	2.000	30,159.293
HHO	70.000	90.000	1.000	699.021	2.000	30,159.289
AOA	70.037	90.111	1.000	1000.000	2.000	30,298.570

**Fig. 16** Car side design problem [101]

other fields, SFOA should be investigated further by adapting different objectives, constraints, or specific cases.

6 Conclusions and future works

This work presents a novel metaheuristic algorithm called the starfish optimization algorithm (SFOA) for optimization problems, inspired by starfish behaviors of exploration, preying, and regeneration. SFOA consists of three phases: initialization, exploration, and exploitation. During initialization, SFOA generates positions randomly within designated variable boundaries. In the exploration phase, SFOA employs a hybrid search pattern combining the five-dimensional and the unidimensional search patterns to achieve high search efficiency and capacity, which is beneficial to solve optimization problems with different dimensions. In the exploitation phase of SFOA, the preying phase designs the parallel two-directional search strategy to update the positions of starfish, and designs the regeneration strategy to promote the global convergence and accelerate the convergence rate during the optimization process. It should be pointed out that the number of function evaluations of SFOA is equal to that of PSO, but the computational efficiency of SFOA is better than that of

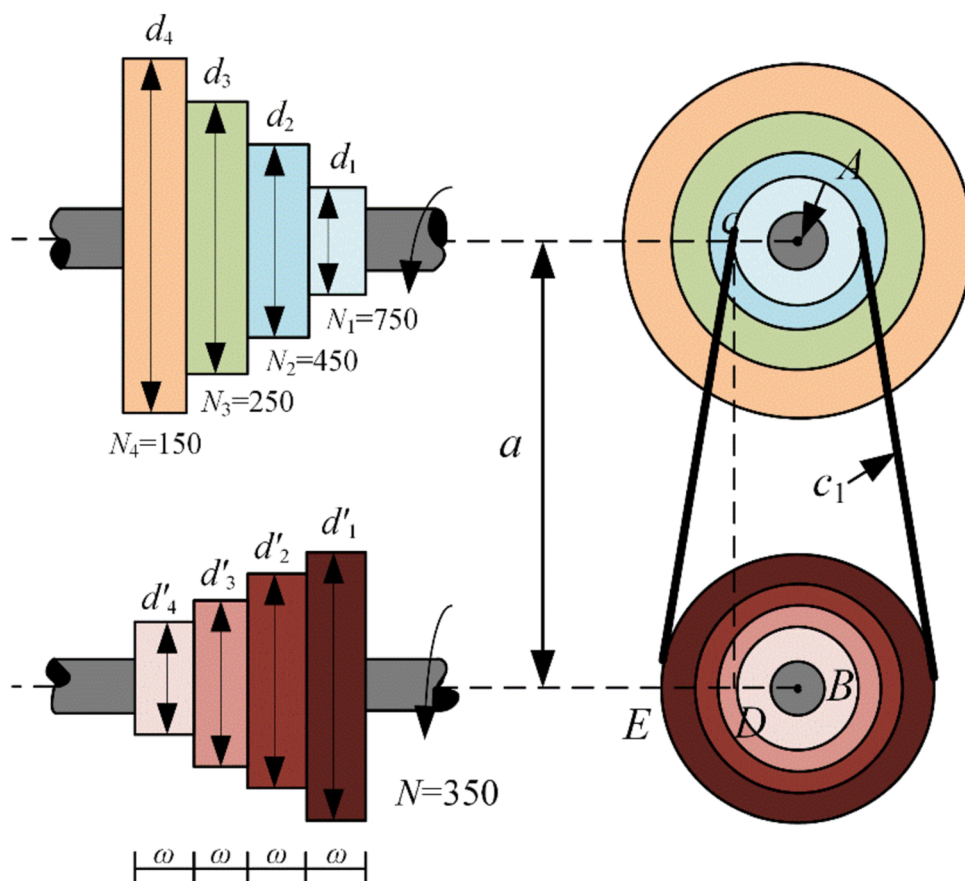
PSO. This illustrates that SFOA is a fast stochastic search optimization algorithm.

We verify the performance of SFOA through a series of experiments of 65 tested benchmark functions, including 24 classical benchmark functions, 29 functions from the CEC 2017 test suite, and 12 functions from the CEC 2022 test suite. We use 100 metaheuristic algorithms to compare with the proposed SFOA in solving benchmark functions, including four algorithms before 2000, 13 algorithms between 2000 and 2009, 29 algorithms in 2010–2019, and 54 algorithms since 2020, which can measure the pros and cons of SFOA, and the performances of the total 100 compared algorithms in solving various benchmark functions. In the quantitative analysis, SFOA provides good performance in solving benchmark functions, including the sensitivity analysis and scalability analysis, which demonstrates its good capacity for some high-dimensional optimization problems. When compared with 100 optimizers, SFOA is outstanding in solving benchmark functions, with 95 wins, two draws and three losses in terms of accuracy, and 97 wins and three losses in terms of efficiency/CPU time, is only worse than WFO both in accuracy and efficiency. Moreover, we introduce the Rank 1 percentage to measure the performance differences between SFOA and all compared algorithms, and the results indicate that SFOA obtains Rank 1 percentages of 85.3%, 86.5%, 87.3% in 24 classical functions, CEC 2017 test suite, and CEC 2022 test suite, respectively, which outperforms the compared algorithms obviously. After that, the convergence analysis also showed that SFOA has good convergence behaviors in the majority of benchmark functions. Finally, SFOA performs well in solving ten real-world optimization problems, illustrating its effectiveness and robustness for real-world applications. According to the detailed experimental verification, it can be concluded that SFOA is highly competitive for global optimization.

Although SFOA has good performance in solving benchmark functions and is successfully applied in real-world optimization problems, it still needs to improve in future study. It is not easy for SFOA to balance the exploration and exploitation capacities for complex

Table 20 Best solutions for car side impact design problem

Algorithm	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	x_{11}	$f_{\min}$
SFOA	0.500	1.234	0.500	1.187	0.875	0.892	0.400	0.345	0.192	1.500	0.572	23.5616
PSO	0.500	1.229	0.500	1.204	0.875	1.200	0.400	0.345	0.192	0.500	1.500	23.5916
BBO	0.663	1.157	0.500	1.211	0.875	0.882	0.400	0.345	0.232	0.726	0.913	23.9386
GWO	0.500	1.229	0.500	1.201	0.875	0.953	0.400	0.345	0.242	0.543	0.774	23.5815
WOA	0.782	1.187	0.633	1.208	0.875	1.046	0.400	0.264	0.204	0.713	0.532	25.6376
SSA	0.500	1.279	0.500	1.184	0.878	0.940	0.414	0.273	0.231	1.220	1.254	23.8918
HHO	0.500	1.263	0.500	1.195	0.875	1.128	0.403	0.321	0.192	0.629	0.500	23.7947
AOA	0.500	1.230	0.500	1.274	0.875	1.200	0.400	0.345	0.196	0.500	0.500	23.8808

Fig. 17 Step cone pulley**Table 21** Best solutions for step cone pulley

Algorithm	x_1	x_2	x_3	x_4	x_5	$f_{\min}$
SFOA	38.4140	52.8586	70.4727	84.4957	90.0000	16.0903
PSO	39.7287	54.6691	72.8863	87.3870	90.0000	17.2108
BBO	38.4644	52.9287	70.5653	84.6068	89.8840	16.1487
GWO	40.5002	55.7263	74.2890	89.0963	86.9421	42.4976
WOA	39.6123	54.5088	72.6726	87.1311	87.2703	16.5915
SSA	39.4224	54.2473	72.3239	86.7134	88.3208	16.6302
HHO	39.6447	54.5534	72.7318	87.2023	87.1988	16.6123
AOA	41.3436	58.0581	75.0738	90.0000	90.0000	26.3444

Table 22 Statistical results for real-world optimization problems

Problems	Algo	SFOA	PSO	BBO	GWO	WOA	SSA	HHO	AOA
E01 Cantilever beam	Min	13.03251	13.03252	13.04115	13.03258	13.21079	13.03285	13.03582	13.48013
	Mean	13.03251	13.03255	13.07725	13.03305	13.93227	13.19284	13.05674	20.78255
	Max	13.03251	13.03270	13.16217	13.03416	15.54236	16.88726	13.07788	36.15874
	STD	1.16E-07	3.85E-05	0.03524	0.00034	0.68301	0.70041	0.01024	5.81531
E02 Welded beam design	Min	1.69525	1.69525	1.75457	1.69585	1.75487	1.69728	1.71760	1.86367
	Mean	1.69526	1.70517	2.21297	1.69679	2.03813	1.83818	1.84666	2.25718
	Max	1.69532	1.79379	3.29528	1.70197	3.08367	2.16415	2.24932	2.90210
	STD	1.29E-05	0.03005	0.34728	0.00110	0.28550	0.12214	0.12036	0.28198
E03 Tension/compression Spring design	Min	0.0126652	0.0126682	0.0126808	0.0126719	0.0126652	0.0126759	0.0126652	0.0131072
	Mean	0.0126652	0.0134064	0.0146231	0.0127304	0.0133830	0.0135089	0.0138157	0.0132180
	Max	0.0126652	0.0304549	0.0170996	0.0131101	0.0163048	0.0167913	0.0164012	0.0134179
	STD	8.38E-09	0.0032253	0.0012403	7.30E-05	0.0010034	0.0010129	0.0010621	5.04E-05
E04 Pressure vessel design	Min	5885.3328	5988.3921	5961.2661	5888.3795	6151.6789	5973.7578	5949.3696	7130.0680
	Mean	5885.3797	6180.2413	6501.8723	5946.5441	7747.5510	7728.4243	6741.2184	38,393.684
	Max	5885.6638	6572.3804	7339.0978	7271.2213	10,538.226	16,910.134	7543.3759	275,484.40
	STD	0.0668	136.6952	318.8462	250.8668	1163.0104	2454.4640	443.5132	55,942.18
E05 Three bar truss	Min	263.89584	263.89584	263.89925	263.89596	263.89794	263.89584	263.89591	263.96878
	Mean	263.89584	263.89598	264.22881	263.89792	264.23814	263.89715	263.94134	264.66361
	Max	263.89584	263.89708	266.93964	263.90774	265.37564	263.90422	264.06695	266.16151
	STD	1.38E-13	0.00023	0.57709	0.00244	0.38075	0.00200	0.05372	0.53743
E06 Speed reducer design	Min	2996.3481	2996.3481	2996.3499	2998.1634	3006.5959	3001.2829	3002.1645	3068.6255
	Mean	2996.3481	3027.2457	2996.3576	3002.3793	3092.9588	3034.6559	3084.8714	3126.4657
	Max	2996.3481	3046.7137	2996.3672	3010.7363	3182.8760	3112.6239	3487.3126	3222.4057
	STD	2.24E-12	18.8942	0.0050	3.3623	59.9908	30.9897	113.5088	42.9657
E07 Himmelblau problem	Min	-30.6655.54	-30.6655.54	-30.640.38	-30.664.81	-30.504.59	-30.652.16	-30.659.89	-30.617.32
	Mean	-30.6655.54	-30.6655.54	-30.473.11	-30.663.22	-29.983.88	-30.522.58	-30.560.59	-29.695.53
	Max	-30.6655.54	-30.6655.54	-30.235.23	-30.661.58	-29.306.52	-30.364.05	-30.224.10	-29.021.29
	STD	3.70E-11	1.48E-11	85.9500	0.9190	295.5895	93.5815	127.9447	354.0884
E08 Multiple disk clutch brake design	Min	30.159.289	30.159.289	30.159.290	30.159.321	30.159.289	30.159.289	30.159.289	30.212.529
	Mean	30.159.289	30.159.289	30.620.812	30.160.912	30.159.289	30.193.870	30.159.289	30.445.113
	Max	30.159.289	30.159.289	32.932.879	30.165.734	30.159.291	30.829.674	30.159.289	34,119.518
	STD	1.11E-11	1.11E-11	769.1878	1.6285	0.0003	126.9185	1.26E-11	706.7594
E09 Car side impact design	Min	23.56158	23.56274	23.56435	23.57368	23.83068	23.59217	23.68606	23.88079
	Mean	23.56160	23.86547	23.74405	23.58583	25.22099	23.98562	24.09400	24.03339
	Max	23.56203	24.31037	25.12907	23.59341	28.32890	25.77538	24.62823	24.10695
	STD	8.23E-05	0.34217	0.33713	0.00594	1.27690	0.41606	0.26905	0.04736

Table 22 (continued)

Problems	Algo	SFOA	PSO	BBO	GWO	WOA	SSA	HHO	AOA
E10	Min	16.09027	16.41467	16.10616	19.82761	16.45692	16.17452	16.56065	23,448
Step cone pulley problem	Mean	16.09027	88,039	18.58379	40.68795	79,594	17.11541	16.75446	164,153
	Max	16.09028	880,225	37.54219	97.04859	1,198,769	18.11737	17.11299	499,130
	STD	4.49E-07	268,576	4.42789	17.62912	247,083	0.53754	0.19135	133,814

constraint optimization problems, so combining with different operators, other metaheuristic algorithms or specific constraint handling techniques may enhance its performance. Moreover, it needs to fully investigate the potential of SFOA for complex or high-dimensional real-world optimization problems in practical engineering with different objectives, constraints, or specific cases.

Funding This work was supported by the National Key Research and Development Program (Grant No. 2019YFA0706803), National Natural Science Foundation of China (Grant No. 12372119, No. 12372195, No. 12402139), Dreams Foundation of Jianghuai Advance Technology Center Foundation (Grant No. 2023-ZM01X013), supported by Hainan Provincial Natural Science Foundation of China (Grant No. 524QN223), supported by Anhui Natural Science Funds for Distinguished Young Scholar (Grant No. 2408085J007), the Scientific Research Startup Foundation of Hainan University (Grant No. RZ2300002710), State Key Laboratory of Structural Analysis, Optimization and CAE Software for Industrial Equipment, Dalian University of Technology (Grant No. GZ24107). Thank you for valuable comments from editors and reviewers to increase the quality of the manuscript. The first author also thanks Wenbo Xue for drawing pictures.

Data availability The source code of SFOA is available at Supplementary Materials, and website: <https://ww2.mathworks.cn/matlabcentral/fileexchange/173735-starfish-optimization-algorithm-sfoa>.

Declarations

Conflict of interest The authors declare that they have no competing financial interest or personal relationships that could have appeared to influence the work reported in this paper.

Supplementary Information

The online version contains supplementary material available at <https://doi.org/10.1007/s00521-024-10694-1>.

References

- Kumar A, Wu G, Ali MZ, Mallipeddi R, Suganthan PN, Das S (2020) A test-suite of non-convex constrained optimization problems from the real-world and some baseline results. *Swarm Evol Comput* 56:100693. <https://doi.org/10.1016/j.swevo.2020.100693>
- Meng Z, Guo L, Wang X (2022) A general fidelity transformation framework for reliability-based design optimization with arbitrary precision. *Struct Multidiscip Optim* 65:14. <https://doi.org/10.1007/s00158-021-03091-y>
- Houssein EH, Oliva D, Çelik E, Emam MM, Choniem RM (2023) Boosted sooty tern optimization algorithm for global optimization and feature selection. *Expert Syst Appl* 213:119015. <https://doi.org/10.1016/j.eswa.2022.119015>
- Liu M, Yao X, Li Y (2020) Hybrid whale optimization algorithm enhanced with Lévy flight and differential evolution for job shop scheduling problems. *Appl Soft Comput* 87:105954. <https://doi.org/10.1016/j.asoc.2019.105954>

5. Li G, Hu H (2014) Risk design optimization using many-objective evolutionary algorithm with application to performance-based wind engineering of tall buildings. *Struct Saf* 48:1–14. <https://doi.org/10.1016/j.strusafe.2014.01.002>
6. Zhong C, Li G (2022) Comprehensive learning Harris hawks-equilibrium optimization with terminal replacement mechanism for constrained optimization problems. *Expert Syst Appl* 192:116432. <https://doi.org/10.1016/j.eswa.2021.116432>
7. Kennedy J, Eberhart RC (1995) Particle swarm optimization. In: *Proceedings of IEEE international conference on neural networks*, Perth, vol 4. pp 1942–1948. <https://doi.org/10.1109/MHS.1995.494215>
8. Holland JH (1975) *Adaptation in natural and artificial systems*, 2nd edn. University of Michigan Press, Ann Arbor
9. Storm R, Price K (1997) Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *J Global Optim* 11:341–359. <https://doi.org/10.1023/A:1008202821328>
10. Lynn N, Suganthan PN (2017) Ensemble particle swarm optimizer. *Appl Soft Comput* 55:533–548. <https://doi.org/10.1016/j.asoc.2017.02.007>
11. Deb K, Pratap A, Meyarivan T (2002) A fast and elitist multi-objective genetic algorithm: NSGA-II. *IEEE Trans Evolut Comput* 6(2):182–197
12. Tanabe R, Fukunaga A (2013) Success-history based parameter adaptation for differential evolution. In: *2013 IEEE congress on evolutionary computation*, June 20–23, Cancun, Mexico, pp 71–78
13. Simon D (2008) Biogeography-based optimization. *IEEE Trans Evol Comput* 12(6):702–713. <https://doi.org/10.1109/TEVC.2008.919004>
14. Mirjalili S, Mirjalili SM, Lewis A (2014) Grey wolf optimizer. *Adv Eng Softw* 69:46–61. <https://doi.org/10.1016/j.advengsoft.2013.12.007>
15. Rao RV, Savsani VJ, Vakharia DP (2011) Teaching-learning-based optimization: a novel method for constrained mechanical design optimization problems. *Comput Aided Des* 43:303–315. <https://doi.org/10.1016/j.cad.2010.12.015>
16. Heidari AA, Mirjalili S, Faris H, Aljarah I, Mafarja M, Chen H (2019) Harris hawks optimization: algorithm and applications. *Futur Gener Comput Syst* 97:849–872. <https://doi.org/10.1016/j.future.2019.02.028>
17. Abualigah L, Diabat A, Mirjalili S, Elaziz MA, Gandomi AH (2021) The arithmetic optimization algorithm. *Comput Methods Appl Mech Eng* 376:113609. <https://doi.org/10.1016/j.cma.2020.113609>
18. Wolpert DH, Macready WG (1997) No free lunch theorem for optimization. *IEEE Trans Evolut Comput* 1(1):67–82. <https://doi.org/10.1109/4235.585893>
19. Sörensen K (2015) Metaheuristics—the metaphor exposed. *Int Trans Oper Res* 22:3–18. <https://doi.org/10.1111/itor.12001>
20. Alam S, Dobbie G, Koh YS, Riddle P, Rehman SU (2014) Research on particle swarm optimization based clustering: a systematic review of literature and techniques. *Swarm Evol Comput* 17:1–13. <https://doi.org/10.1016/j.swevo.2014.02.001>
21. Nayak J, Swapnarekha H, Naik B, Ghiman G, Vimal S (2023) 25 years of particle swarm optimization: flourishing voyage of two decades. *Arch Comput Methods Eng* 30:1663–1725. <https://doi.org/10.1007/s11831-022-09849-x>
22. Dorigo M, Maniezzo V, Colomi A (1996) Any system: optimization by a colony of cooperating agents. *IEEE Trans Syst Man Cybern Part B* 26(1):29–41. <https://doi.org/10.1109/3477.484436>
23. Karaboga D, Basturk B (2007) A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. *J Global Optim* 39:459–471. <https://doi.org/10.1007/s10898-007-9149-x>
24. Mirjalili S, Gandomi AH, Mirjalili SZ, Saremi S, Faris H, Mirjalili SM (2017) Salp Swarm algorithm: a bio-inspired optimizer for engineering design problems. *Adv Eng Softw* 114:163–191. <https://doi.org/10.1016/j.advengsoft.2017.07.002>
25. Faramarzi A, Heidarinejad M, Mirjalili S (2020) Marine predators algorithm: a nature-inspired metaheuristic. *Expert Syst Appl* 152:113377. <https://doi.org/10.1016/j.eswa.2020.113377>
26. Zhong C, Li G, Meng Z (2022) Beluga whale optimization: a novel nature-inspired metaheuristic algorithm. *Knowl-Based Syst* 251:109215. <https://doi.org/10.1016/j.knsys.2022.109215>
27. Eusuff M, Lansey K, Pasha F (2006) Shuffled frog-leaping algorithm: a memetic meta-heuristic for discrete optimization. *Eng Optim* 38(2):129–154. <https://doi.org/10.1080/03052150500384759>
28. Yang X-S (2008) *Nature-inspired metaheuristic algorithm*. Luniver Press, Bristol
29. Yang X-S, Deb S (2009) Cuckoo search via Lévy flights. In: *proceedings of the 2009 IEEE world congress on nature & biologically inspired computing*, NaBIC 2009, pp 210–214
30. Yang X-S, Gandomi AH (2012) Bat algorithm: a novel approach for global engineering optimization. *Eng Comput* 29(5):464–483. <https://doi.org/10.1108/02644401211235834>
31. Yang X-S (2012) Flower pollination algorithm for global optimization. In: *Unconventional computation and natural computation*. Springer, Berlin, pp 240–249
32. Gandomi AH, Alavi AH (2012) Krill herd: a new bio-inspired optimization algorithm. *Commun Nonlinear Sci Numer Simul* 17(12):4831–4845. <https://doi.org/10.1016/j.cnsns.2012.05.010>
33. Uymaz SA, Yezel G, Yel E (2015) Artificial algae algorithm (AAA) for nonlinear global optimization. *Appl Soft Comput* 31:153–171. <https://doi.org/10.1016/j.asoc.2015.03.003>
34. Mirjalili S (2015) Moth-flame optimization algorithm: a novel nature-inspired heuristic paradigm. *Knowl-Based Syst* 89:228–249. <https://doi.org/10.1016/j.knsys.2015.07.006>
35. Mirjalili S, Lewis A (2016) The whale optimization algorithm. *Adv Eng Softw* 95:51–67. <https://doi.org/10.1016/j.advengsoft.2016.01.008>
36. Dhiman G, Kumar V (2017) Spotted hyena optimizer: a novel bio-inspired based metaheuristic technique for engineering applications. *Adv Eng Softw* 114:48–70. <https://doi.org/10.1016/j.advengsoft.2017.05.014>
37. Dhiman G, Kumar V (2019) Seagull optimization algorithm: theory and its applications for large-scale industrial engineering problems. *Knowl-Based Syst* 165:169–196. <https://doi.org/10.1016/j.knsys.2018.11.024>
38. Shadravan S, Naji HR, Bardsiri VK (2019) The sailfish optimizer: a novel nature-inspired metaheuristic algorithm for solving constrained engineering optimization problems. *Eng Appl Artif Intell* 80:20–34. <https://doi.org/10.1016/j.engappai.2019.01.001>
39. Alsattar HA, Zaidan AA, Zaidan BB (2020) Novel metaheuristic bald eagle search optimization algorithm. *Artif Intell Rev* 53:2237–2264. <https://doi.org/10.1007/s10462-019-09732-5>
40. Khishe M, Mosavi MR (2020) Chimp optimization algorithm. *Expert Syst Appl* 149:113338. <https://doi.org/10.1016/j.eswa.2020.113338>
41. Zhao W, Zhang Z, Wang L (2020) Manta ray foraging optimization: an effective bio-inspired optimizer for engineering applications. *Eng Appl Artif Intell* 87:103300. <https://doi.org/10.1016/j.engappai.2019.103300>
42. Li S, Chen H, Wang M, Heidari AA, Mirjalili S (2020) Slime mould algorithm: a new method for stochastic optimization.

- Futur Gener Comput Syst 111:300–323. <https://doi.org/10.1016/j.future.2020.03.055>
43. Kaur S, Awasthi LK, Sangal AL, Dhiman G (2020) Tunicate swarm algorithm: a new bio-inspired based metaheuristic paradigm for global optimization. *Eng Appl Artif Intell* 90:103541. <https://doi.org/10.1016/j.engappai.2020.103541>
 44. Abualigah L, Yousri D, Elaziz MA, Ewees AA, Al-qaness MAA, Gandomi AH (2021) Aquila optimizer: a novel metaheuristic optimization algorithm. *Comput Ind Eng* 157:107250. <https://doi.org/10.1016/j.cie.2021.107250>
 45. Meng OK, Pauline O, Kiong SC (2021) A carnivorous plant algorithm for solving global optimization problems. *Appl Soft Comput* 98:106833. <https://doi.org/10.1016/j.asoc.2020.106833>
 46. Braik MS (2021) Chameleon swarm algorithm: a bio-inspired optimizer for solving engineering design problems. *Expert Syst Appl* 174:114685. <https://doi.org/10.1016/j.eswa.2021.114685>
 47. Mohammadi-Balani A, Nayeri MD, Azar A, Taghizadeh-Yazdi M (2021) Golden eagle optimizer: a nature-inspired metaheuristic algorithm. *Comput Ind Eng* 152:107050. <https://doi.org/10.1016/j.cie.2020.107050>
 48. MiarNaeimi F, Azizyan G, Rashki M (2021) Horse herd optimization algorithm: a nature-inspired algorithm for high-dimensional optimization problems. *Knowl-Based Syst* 213:106711. <https://doi.org/10.1016/j.knsys.2020.106711>
 49. Połap D, Woźniak M (2021) Red fox optimization algorithm. *Expert Syst Appl* 166:114107. <https://doi.org/10.1016/j.eswa.2020.114107>
 50. Zhao W, Wang L, Mirjalili S (2022) Artificial hummingbird algorithm: a new bio-inspired optimizer with its engineering applications. *Comput Methods Appl Mech Eng* 388:114914. <https://doi.org/10.1016/j.cma.2021.114194>
 51. Wang L, Cao Q, Zhang Z, Mirjalili S, Zhao W (2022) Artificial rabbits optimization: a new bio-inspired meta-heuristic algorithm for solving engineering optimization problems. *Eng Appl Artif Intell* 114:105082. <https://doi.org/10.1016/j.engappai.2022.105082>
 52. Akbari MA, Azizipناه-abarghoee R, Mirjalili S, Deriche M (2022) The cheetah optimizer: a nature-inspired metaheuristic algorithm for large-scale optimization problems. *Sci Rep* 12:10953. <https://doi.org/10.1038/s41598-022-14338-z>
 53. Zhao S, Zhang T, Ma S, Chen M (2022) Dandelion optimizer: a nature-inspired metaheuristic algorithm for engineering applications. *Eng Appl Artif Intell* 114:105075. <https://doi.org/10.1016/j.engappai.2022.105075>
 54. Agushaka JO, Ezugwu AE, Abualigah L (2022) Dwarf mon-goose optimization algorithm. *Comput Methods Appl Mech Eng* 391:114570. <https://doi.org/10.1016/j.cma.2022.114570>
 55. Chopra N, Ansari MM (2022) Golden jackal optimization: a novel nature-inspired optimizer engineering applications. *Expert Syst Appl* 198:116924. <https://doi.org/10.1016/j.eswa.2022.116924>
 56. Pan JS, Zhang LG, Wang RB, Snášel V, Chu SC (2022) Gannet optimization algorithm: a new metaheuristic algorithm for solving engineering optimization problems. *Math Comput Simul* 202:343–373. <https://doi.org/10.1016/j.matcom.2022.06.007>
 57. Hashim FA, Houssein EH, Hussain K, Mabrouk MS, Al-Atabany W (2022) Honey badger algorithm: new metaheuristic algorithm for solving optimization problems. *Math Comput Simul* 192:84–110. <https://doi.org/10.1016/j.matcom.2021.08.013>
 58. Suyanto S, Ariyanto AA, Ariyanto AF (2022) Komodo Mlipir algorithm. *Appl Soft Comput* 114:108043. <https://doi.org/10.1016/j.asoc.2021.108043>
 59. Abdollahzadeh B, Gharehchopogh FS, Khodadadi N, Mirjalili S (2022) Mountain Gazelle optimizer: a new nature-inspired metaheuristic algorithm for global optimization problems. *Adv Eng Softw* 174:103282. <https://doi.org/10.1016/j.advengsoft.2022.103282>
 60. Jiang Y, Wu Q, Zhu S, Zhang L (2022) Orca predation algorithm: a novel bio-inspired algorithm for global optimization problems. *Expert Syst Appl* 188:116026. <https://doi.org/10.1016/j.eswa.2021.116026>
 61. Ezugwu AE, Agushaka JO, Abualigah L, Mirjalili S, Gandomi AH (2022) Prairie dog optimization algorithm. *Neural Comput Appl* 34:20017–20065. <https://doi.org/10.1007/s00521-022-07530-9>
 62. Abualigah L, Elaziz MA, Sumari P, Geem ZW, Gandomi AH (2022) Reptile search algorithm (RSA): a nature-inspired metaheuristic optimizer. *Expert Syst Appl* 191:116158. <https://doi.org/10.1016/j.eswa.2021.116158>
 63. Hashim FA, Hussien AG (2022) Snake optimizer: a novel metaheuristic optimization algorithm. *Knowl-Based Syst* 242:108320. <https://doi.org/10.1016/j.knsys.2022.108320>
 64. Zamani H, Nadimi-Shahraki MH, Gandomi AH (2022) Starling murmuration optimizer: a novel bio-inspired algorithm for global and engineering optimization. *Comput Methods Appl Mech Eng* 392:114616. <https://doi.org/10.1016/j.cma.2022.114616>
 65. Braik M, Hammouri A, Atwan J, Al-Betar MA, Awadallah MA (2022) White shark optimizer: a novel bio-inspired metaheuristic algorithm for global optimization problems. *Knowl-Based Syst* 243:108457. <https://doi.org/10.1016/j.knsys.2022.108457>
 66. Jia H, Rao H, Wen C, Mirjalili S (2023) Crayfish optimization algorithm. *Artif Intell Rev* 56:1919–1979. <https://doi.org/10.1007/s10462-023-10567-4>
 67. Seyyedabbasi A, Kiani F (2023) Sand cat swarm optimization: a nature-inspired algorithm to solve global optimization problems. *Eng Comput* 39:2627–2651. <https://doi.org/10.1007/s00366-022-01604-x>
 68. Minh H, Sang-To T, Theraulaz G, Wahab MA, Cuong-Le T (2023) Termite life cycle optimizer. *Expert Syst Appl* 213:119211. <https://doi.org/10.1016/j.eswa.2022.119211>
 69. Zhong C, Li G, Meng Z, Li H, He W (2023) Multi-objective SHADE with manta ray foraging optimizer for structural design problems. *Appl Soft Comput* 134:110016. <https://doi.org/10.1016/j.asoc.2023.110016>
 70. Awad NH, Ali MZ, Suganthan PN (2017) Ensemble sinusoidal differential covariance matrix adaptation with Euclidean neighborhood for solving CEC 2017 benchmark problems. In: *proceedings of 2017 IEEE congress on evolutionary computation (CEC)*, June 05–08, Donostia, Spain, pp 372–379
 71. Hansen N, Müller SD, Koumoutsakos P (2003) Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES). *Evol Comput* 11(1):1–18
 72. Koza JR (1992) Genetic programming, on the programming of computers by means of natural selection. MIT Press, Cambridge
 73. Lewis A, Abramson D, Peachey T (2004) An evolutionary programming algorithm for automatic engineering design. In: *proceedings of PPAM 2003: parallel processing and applied mathematics*, pp 586–594
 74. Sulaiman MH, Mustafa Z, Saari MM, Daniyal H (2020) Barnacles mating optimizer: a new bio-inspired algorithm for solving engineering optimization problems. *Eng Appl Artif Intell* 87:103330. <https://doi.org/10.1016/j.engappai.2019.103330>
 75. Luo K (2022) Water flow optimizer: a nature-inspired evolutionary algorithm for global optimization. *IEEE Trans Cybern* 52(8):7753–7764. <https://doi.org/10.1109/TCYB.2021.3049607>
 76. Lan G, Hooft M, De Carlo M, Tomczak JM, Eiben AE (2021) Learning locomotion skills in evolvable robots. *Neurocomputing* 452:294–306. <https://doi.org/10.1016/j.neucom.2021.03.030>

77. Lan G, Tomczak JM, Roijers DM, Eiben AE (2022) Time efficiency in optimization with a Bayesian-evolutionary algorithm. *Swarm Evolut Comput* 69:100970. <https://doi.org/10.1016/j.swevo.2021.100970>
78. Kirkpatrick S, Gelatt CD Jr, Vecchi MP (1983) Optimization by simulated annealing. *Science* 220(4598):671–680. <https://doi.org/10.1126/science.220.4598.671>
79. Erol OK, Eksin I (2006) A new optimization method: big bang-big crunch. *Adv Eng Softw* 37:106–111. <https://doi.org/10.1016/j.advengsoft.2005.04.005>
80. Rashedi E, Nezamabadi-pour H, Saryazdi S (2009) GSA: a gravitational search algorithm. *Inf Sci* 179:2232–2248. <https://doi.org/10.1016/j.ins.2009.03.004>
81. Kaveh A, Talatahari S (2010) A novel heuristic optimization method: charged system search. *Acta Mech* 213:267–289. <https://doi.org/10.1007/s00707-009-0270-4>
82. Eskandar H, Sadollah A, Bahreininejad A, Hamdi M (2012) Water cycle algorithm—a novel metaheuristic optimization method for solving constrained engineering optimization problems. *Comput Struct* 110–111:151–166. <https://doi.org/10.1016/j.compstruc.2012.07.010>
83. Faramarzi A, Heidarinejad M, Stephens B, Mirjalili S (2020) Equilibrium optimizer: a novel optimization algorithm. *Knowl-Based Syst* 191:105190. <https://doi.org/10.1016/j.knosys.2019.105190>
84. Mehrabian AR, Lucas C (2006) A novel numerical optimization algorithm inspired from weed colonization. *Eco Inform* 4(1):355–366. <https://doi.org/10.1016/j.ecoinf.2006.07.003>
85. Tan Y, Zhu Y (2010) Fireworks algorithm for optimization. In: *Proceedings of ICSI 2010. Lecture notes in computer science*, vol 6145. Springer, Berlin, Heidelberg, pp 355–364. https://doi.org/10.1007/978-3-642-13495-1_44
86. Kaveh A, Khayatizad M (2012) A new meta-heuristic method: ray optimization. *Comput Struct* 112–113:283–294. <https://doi.org/10.1016/j.compstruc.2012.09.003>
87. Kaveh A, Mahdavi VR (2014) Colliding bodies optimization: a novel meta-heuristic method. *Comput Struct* 139:18–27. <https://doi.org/10.1016/j.compstruc.2014.04.005>
88. Salimi H (2015) Stochastic fractal search: a powerful meta-heuristic algorithm. *Knowl-Based Syst* 75:1–18. <https://doi.org/10.1016/j.knosys.2014.07.025>
89. Mirjalili S, Mirjalili SM, Hatamlou A (2016) Multi-verse optimizer: a nature-inspired algorithm for global optimization. *Neural Comput Appl* 27:495–513. <https://doi.org/10.1007/s00521-015-1870-7>
90. Mirjalili S (2016) SCA: a sine cosine algorithm for solving optimization problems. *Knowl-Based Syst* 96:120–133. <https://doi.org/10.1016/j.knosys.2015.12.022>
91. Kaveh S, Bakhshpoori T (2016) Water evaporation optimization: a novel physically inspired optimization algorithm. *Comput Struct* 167:69–85. <https://doi.org/10.1016/j.compstruc.2016.01.008>
92. Kaveh A, Dardas A (2017) A novel meta-heuristic optimization algorithm: thermal exchange optimization. *Adv Eng Softw* 110:69–84. <https://doi.org/10.1016/j.advengsoft.2017.03.014>
93. Zhao W, Wang L, Zhang Z (2019) Atom search optimization and its application to solve a hydrogeologic parameter estimation problem. *Knowl-Based Syst* 163:283–304. <https://doi.org/10.1016/j.knosys.2018.08.030>
94. Azizi M (2021) Atomic orbital search: a novel metaheuristic algorithm. *Appl Math Model* 93:657–683. <https://doi.org/10.1016/j.apm.2020.12.021>
95. Zhao W, Wang L, Zhang Z (2020) Artificial ecosystem-based optimization: a novel nature-inspired meta-heuristic algorithm. *Neural Comput Appl* 32:9383–9425. <https://doi.org/10.1007/s00521-019-04452-x>
96. Ghafil HN, Jármai K (2020) Dynamic differential annealed optimization: new metaheuristic optimization algorithm for engineering applications. *Appl Soft Comput* 93:106392. <https://doi.org/10.1016/j.asoc.2020.106392>
97. Houssein EH, Saad MR, Hashim FA, Shaban H, Hassaballah M (2020) Lévy flight distribution: a new metaheuristic algorithm for solving engineering optimization problems. *Eng Appl Artif Intell* 94:103731. <https://doi.org/10.1016/j.engappai.2020.103731>
98. Pereira JLJ, Francisco MB, Diniz CA, Oliver GA, Cunha SS Jr, Gomes GF (2021) Lichtenberg algorithm: a novel hybrid physics-based meta-heuristic for global optimization. *Expert Syst Appl* 170:114522. <https://doi.org/10.1016/j.eswa.2020.114522>
99. Mahdavi-Meymand A, Zounemat-Kermani M (2022) Homonuclear molecules optimization (HMO) meta-heuristic algorithm. *Knowl-Based Syst* 258:110032. <https://doi.org/10.1016/j.knosys.2022.110032>
100. Emami H (2022) Seasons optimization algorithm. *Eng Comput* 38:1845–1865. <https://doi.org/10.1007/s00366-020-01133-5>
101. Goodarzimehr V, Talatahari S, Shojaei S, Hamzehei-Javaran S (2023) Special relativity search for applied mechanics and engineering. *Comput Methods Appl Mech Eng* 403:115734. <https://doi.org/10.1016/j.cma.2022.115734>
102. Hashim FA, Mostafa RR, Hussien AG, Mirjalili S, Sallam KM (2023) Fick's law algorithm: a physical law-based algorithm for numerical optimization. *Knowl-Based Syst* 260:110146. <https://doi.org/10.1016/j.knosys.2022.110146>
103. Su H, Zhao D, Heidari AA, Liu L, Zhang X, Mafarja M, Chen H (2023) RIME: a physics-based optimization. *Neurocomputing* 532:183–214. <https://doi.org/10.1016/j.neucom.2023.02.010>
104. Abdel-Basset M, El-Shahat D, Jameel M, Abouhawwash M (2023) Young's double-slit experiment optimizer: a novel metaheuristic optimization algorithm for global and constraint optimization problems. *Comput Methods Appl Mech Eng* 403:115652. <https://doi.org/10.1016/j.cma.2022.115652>
105. Geem ZW, Kim JH (2001) A new heuristic optimization algorithm: harmony search. *SIMULATION* 76(2):60–68. <https://doi.org/10.1177/003754970107600201>
106. Shi Y (2011) An optimization algorithm based on brainstorming process. *Int J Swarm Intell Res* 2(4):35–62. <https://doi.org/10.4018/jsir.2011100103>
107. Atashpax-Gargari E, Lucas C (2007) Imperialist competitive algorithm: an algorithm for optimization inspired by imperialistic competition. In: *Proceedings of 2007 IEEE congress on evolutionary computation, CEC 2007*, pp 4661–4667. <https://doi.org/10.1109/CEC.2007.4425083>
108. Rao RV (2016) Jaya: a simple and new optimization algorithm for solving constrained and unconstrained optimization problems. *Int J Ind Eng Comput* 7(1):19–34
109. Zhang Y, Chi A, Mirjalili S (2021) Enhanced Jaya algorithm: a simple but efficient optimization method for constrained engineering design problems. *Knowl-Based Syst* 233:107555. <https://doi.org/10.1016/j.knosys.2021.107555>
110. Punathanam V, Kotecha P (2016) Yin–Yang-pair optimization: a novel lightweight optimization algorithm. *Eng Appl Artif Intell* 54:62–79. <https://doi.org/10.1016/j.engappai.2016.04.004>
111. Reynolds RG (1994) An introduction to cultural algorithms. In: *Proceedings of the 3rd annual conference on evolutionary programming*, World Scientific Publishing, pp 131–139
112. Kashan AH (2014) League championship algorithm (LCA): an algorithm for global optimization inspired by sport championships. *Appl Soft Comput* 16:171–200. <https://doi.org/10.1016/j.asoc.2013.12.005>
113. Moosavian N, Roodsari BK (2014) Soccer league competition algorithm: a novel meta-heuristic algorithm for optimal design

- of water distribution networks. *Swarm Evol Comput* 17:14–24. <https://doi.org/10.1016/j.swevo.2014.02.002>
114. Kumar M, Kulkarni AJ, Satapathy SC (2018) Socio evolution & learning optimization algorithm: a socio-inspired optimization methodology. *Futur Gener Comput Syst* 81:252–272. <https://doi.org/10.1016/j.future.2017.10.052>
 115. Zhao W, Wang L, Zhang Z (2019) Supply-demand-based optimization: a novel economics-inspired algorithm for global optimization. *IEEE Access* 7:73182–73206. <https://doi.org/10.1109/ACCESS.2019.2918753>
 116. Kahraman HT, Aaras S, Gedikli E (2020) Fitness-distance balance (FDB): a new selection method for meta-heuristic search algorithm. *Knowl-Based Syst* 190:105169. <https://doi.org/10.1016/j.knosys.2019.105169>
 117. Ahmadianfar I, Bozorg-Haddad O, Chu X (2020) Gradient-based optimizer: a new metaheuristic optimization algorithm. *Inf Sci* 540:131–159. <https://doi.org/10.1016/j.ins.2020.06.037>
 118. Zhang Y, Jin Z (2020) Group teaching optimization algorithm: a novel metaheuristic method for solving global optimization problems. *Expert Syst Appl* 148:113246. <https://doi.org/10.1016/j.eswa.2020.113246>
 119. Askari Q, Saeed M, Younas I (2020) Heap-based optimizer inspired by corporate rank hierarchy for global optimization. *Expert Syst Appl* 161:113702. <https://doi.org/10.1016/j.eswa.2020.113702>
 120. Talatahari S, Azizi M (2021) Chaos game optimization: a novel metaheuristic algorithm. *Artif Intell Rev* 54:917–1004. <https://doi.org/10.1007/s10462-020-09867-w>
 121. Feng Z, Niu W, Liu S (2021) Cooperation search algorithm: a novel metaheuristic evolutionary intelligence algorithm for numerical optimization and engineering optimization problems. *Appl Soft Comput J* 98:106734. <https://doi.org/10.1016/j.asoc.2020.106734>
 122. Yang Y, Chen H, Heidari AA, Gandomi AH (2021) Hunger games search: visions, conception, implementation, deep analysis, perspectives, and towards performance shifts. *Expert Syst Appl* 177:114864. <https://doi.org/10.1016/j.eswa.2021.114864>
 123. Ahmadianfar I, Heidari AA, Gandomi AH, Chu X, Chen H (2021) RUN beyond the metaphor: an efficient optimization algorithm based on Runge Kutta method. *Expert Syst Appl* 181:115079. <https://doi.org/10.1016/j.eswa.2021.115079>
 124. Braik M, Ryalat MH, Al-Zoubi H (2022) A novel meta-heuristic algorithm for solving numerical optimization problems: Ali Baba and the forty thieves. *Neural Comput Appl* 34:409–455. <https://doi.org/10.1007/s00521-021-06392-x>
 125. Yuan Y, Ren J, Wang S, Wang Z, Mu X, Zhao W (2022) Alpine skiing optimization: a new bio-inspired optimization algorithm. *Adv Eng Softw* 170:103158. <https://doi.org/10.1016/j.advengsoft.2022.103158>
 126. Kazemi MV, Veysari EF (2022) A new optimization algorithm inspired by the quest for the evolution of human society: human felicity algorithm. *Expert Syst Appl* 193:116468. <https://doi.org/10.1016/j.eswa.2021.116468>
 127. Ma Z, Wu G, Suganthan PN, Song A, Luo Q (2023) Performance assessment and exhaustive listing of 500+ nature-inspired metaheuristic algorithms. *Swarm Evol Comput* 77:101248. <https://doi.org/10.1016/j.swevo.2023.101248>
 128. Kettle BT, Lucas JS (1987) Biometric relationships between organ indices, fecundity, oxygen consumption and body size in *Acanthaster planci* (L.) (Echinodermata; Asteroidea). *Bull Mar Sci* 41(2):541–551
 129. Ottesen PO, Lucas JS (1982) Divide or broadcast: interrelation of asexual and sexual reproduction in a population of the fissiparous hermaphroditic seastar *Nepanthia belcheri* (Asteroidea: Asterinidae). *Mar Biol* 69:223–233. <https://doi.org/10.1007/BF00397488>
 130. Güler M, Lök A (2015) Foraging behaviors of sea stars, *Marthasterias glacialis* and *Astropecten aranciatus* (Asteroidea) and predator-prey interactions with warty venus clam, *Venus verrucosa* (Bivalvia). *J Exp Mar Biol Ecol* 465:99–106. <https://doi.org/10.1016/j.jembe.2014.12.018>
 131. Hayashi Y, Motokawa T (1986) Effects of ionic environment on viscosity of catch connective tissue in Holothurian body wall. *J Exp Biol* 125(1):71–84. <https://doi.org/10.1242/jeb.125.1.71>
 132. Luo W, Lin X, Li C, Yang S, Shi Y (2022) Benchmark functions for CEC 2022 competition on seeking multiple optima in dynamic environments, arXiv preprint arXiv: 2201.00523
 133. Piotrowski AP, Napiorkowski JJ, Piotrowska AE (2023) Choice of benchmark optimization problems does matter. *Swarm Evol Comput* 83:101378. <https://doi.org/10.1016/j.swevo.2023.101378>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Changting Zhong^{1,2} · Gang Li^{2,3}  · Zeng Meng^{4,5} · Haijiang Li⁶ · Ali Riza Yildiz⁷ · Seyedali Mirjalili^{8,9}

✉ Gang Li
ligang@dlut.edu.cn

✉ Zeng Meng
mengz@hfut.edu.cn

Changting Zhong
zhongct@hainanu.edu.cn

Haijiang Li
lih@cardiff.ac.uk

Ali Riza Yildiz
aliriza@uludag.edu.tr

Seyedali Mirjalili
ali.mirjalili@gmail.com

³ Ningbo Institute of Dalian University of Technology, No. 26, Yucai Road, Jiangbei District, Ningbo 315016, China

⁴ School of Civil Engineering, Hefei University of Technology, Hefei 230009, China

⁵ Jianghuai Advance Technology Center, Hefei 230000, China

⁶ BIM for Smart Engineering Centre, Cardiff School of Engineering, Cardiff University, Queen's Buildings, Cardiff, Wales CF24 3AA, UK

⁷ Department of Mechanical Engineering, College of Engineering, Bursa Uludağ University, Görükle, 16059 Bursa, Turkey

⁸ Centre for Artificial Intelligence Research and Optimisation, Torrens University Australia, Brisbane, Australia

⁹ University Research and Innovation Center, Obuda University, Budapest 1034, Hungary

¹ School of Civil Engineering and Architecture, Hainan University, Haikou 570228, China

² Department of Engineering Mechanics, State Key Laboratory of Structural Analysis Optimization and CAE Software for Industrial Equipment, Dalian University of Technology, Dalian 116024, China